

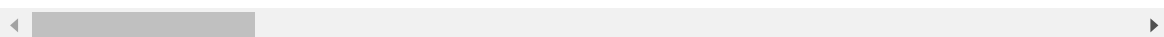
```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.impute import KNNImputer
```

```
In [2]: df = pd.read_csv("C:/Users/THUSHAN/Desktop/competition/fraud_detection_data.csv")
df.head()
```

```
Out[2]:
```

	account_open_date	age	location	occupation	income_level	fico_score	delinquency_status
0	11/9/2023	56	Los Angeles	Teacher	40099	424.0	10
1	9/11/2022	69	New York	Engineer	2050	483.0	
2	7/12/2020	46	Miami	Engineer	71936	566.0	
3	8/13/2024	32	Houston	Banker	15833	NaN	9
4	7/27/2024	60	Houston	Lawyer	8574	787.0	

5 rows × 23 columns



```
In [3]: df.shape
```

```
Out[3]: (7000, 23)
```

```
In [4]: df.dtypes
```

```
Out[4]: account_open_date    object
age                          int64
location                     object
occupation                   object
income_level                 int64
fico_score                   float64
delinquency_status          int64
charge_off_status           bool
number_of_credit_applications int64
debt_to_income_ratio        float64
payment_methods_high_risk   bool
max_balance                 float64
avg_balance_last_12months   float64
number_of_delinquent_accounts float64
number_of_defaulted_accounts int64
earliest_credit_account     object
recent_trade_activity       object
new_accounts_opened_last_12months int64
multiple_applications_short_time_period bool
unusual_submission_pattern  object
applications_submitted_during_odd_hours bool
watchlist_blacklist_flag    bool
public_records_flag         bool
dtype: object
```

```
In [5]: # Convert object columns into date type
date_columns = ['account_open_date', 'earliest_credit_account', 'recent_trade']
df[date_columns] = df[date_columns].apply(pd.to_datetime, errors='coerce')
```

```
In [6]: # Convert unusual_submission_pattern column type into bool type
df['unusual_submission_pattern'] = df['unusual_submission_pattern'].astype(bool)
```

```
In [7]: df.dtypes
```

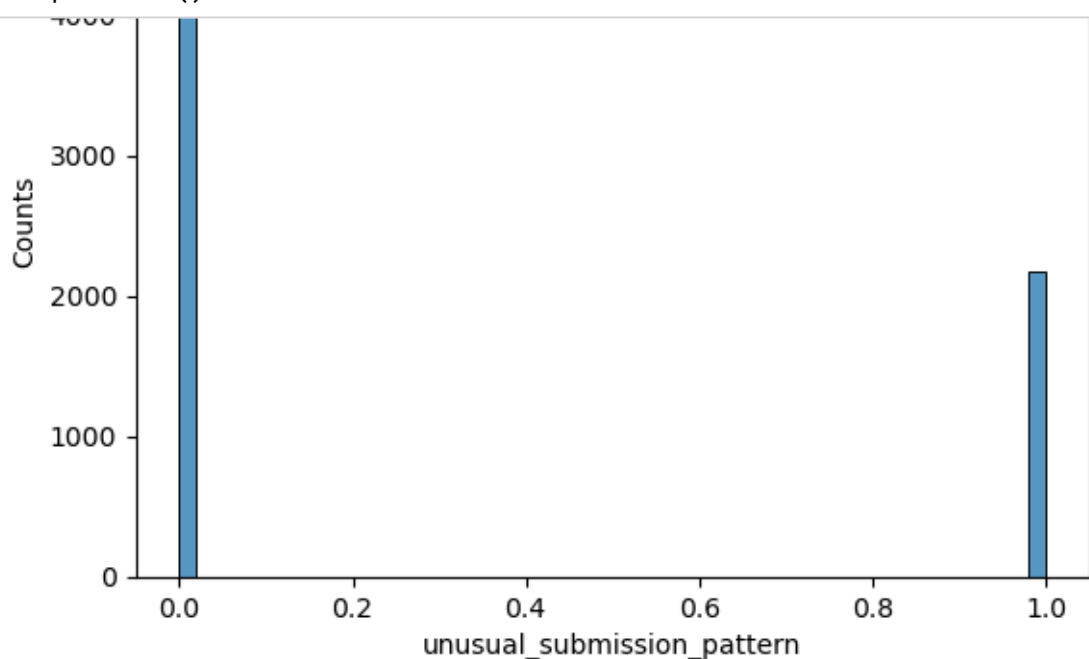
```
Out[7]: account_open_date      datetime64[ns]
age                             int64
location                       object
occupation                     object
income_level                   int64
fico_score                     float64
delinquency_status             int64
charge_off_status              bool
number_of_credit_applications  int64
debt_to_income_ratio           float64
payment_methods_high_risk      bool
max_balance                    float64
avg_balance_last_12months      float64
number_of_delinquent_accounts  float64
number_of_defaulted_accounts   int64
earliest_credit_account        datetime64[ns]
recent_trade_activity          datetime64[ns]
new_accounts_opened_last_12months int64
multiple_applications_short_time_period bool
unusual_submission_pattern      bool
applications_submitted_during_odd_hours bool
watchlist_blacklist_flag        bool
public_records_flag             bool
dtype: object
```

```
In [8]: df.isnull().sum()
```

```
Out[8]: account_open_date      0
age                            0
location                      0
occupation                   0
income_level                 0
fico_score                   210
delinquency_status          0
charge_off_status           0
number_of_credit_applications 0
debt_to_income_ratio        0
payment_methods_high_risk    0
max_balance                 0
avg_balance_last_12months    350
number_of_delinquent_accounts 700
number_of_defaulted_accounts 0
earliest_credit_account      0
recent_trade_activity        0
new_accounts_opened_last_12months 0
multiple_applications_short_time_period 0
unusual_submission_pattern    0
applications_submitted_during_odd_hours 0
watchlist_blacklist_flag     0
public_records_flag         0
dtype: int64
```

```
In [9]: missing_values_columns = ['fico_score', 'avg_balance_last_12months', 'number_
```

```
In [10]: plt.figure(figsize=(10, 5))
for i in range(len(missing_values_columns)):
    sns.histplot(df[missing_values_columns[i]], bins=50)
    plt.title(f"Distribution of {missing_values_columns[i]} Data ")
    plt.xlabel(missing_values_columns[i])
    plt.ylabel('Counts')
    plt.show()
```



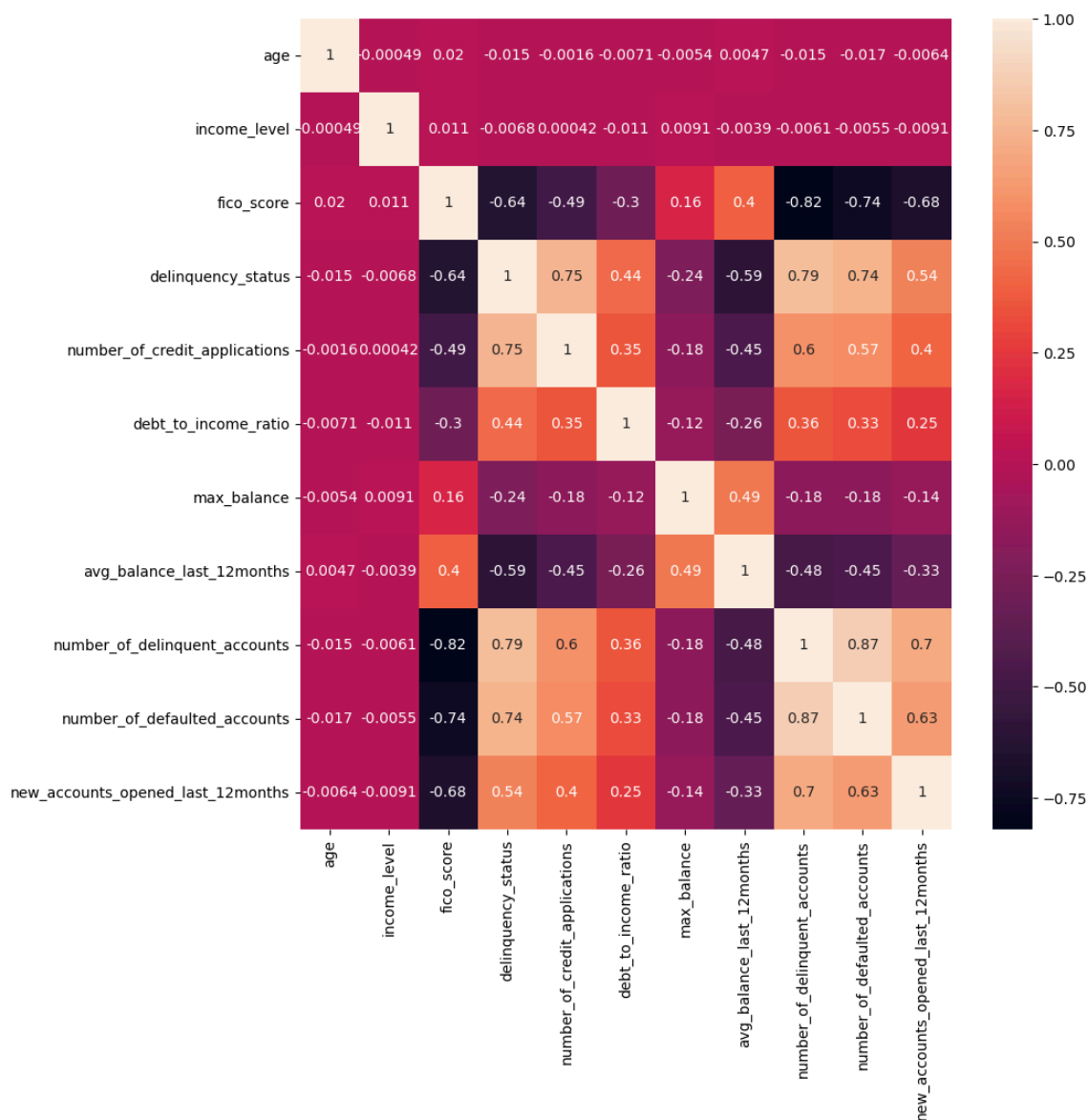
```
In [11]: numeric_df = df.select_dtypes(include=["float64", "int64"])
numeric_df.head()
```

Out[11]:

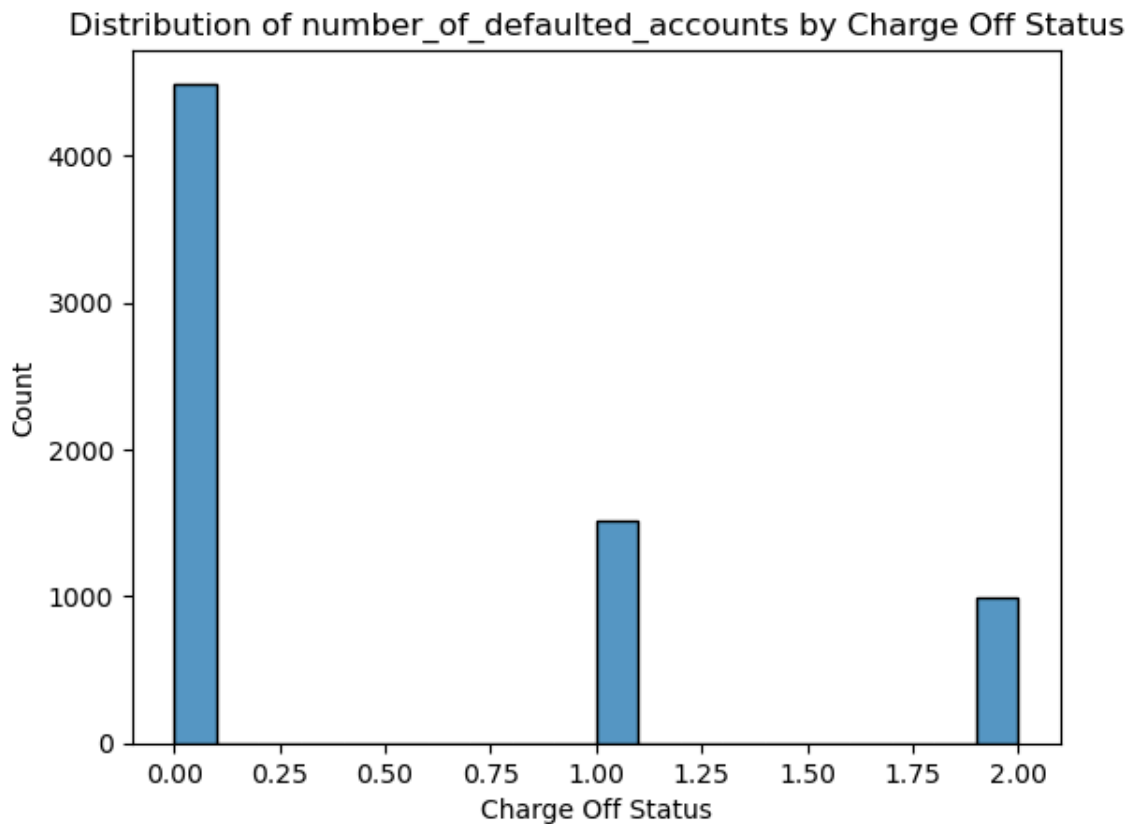
	age	income_level	fico_score	delinquency_status	number_of_credit_applications	debt_to_
0	56	40099	424.0	108		10
1	69	2050	483.0	0		1
2	46	71936	566.0	0		1
3	32	15833	NaN	97		5
4	60	8574	787.0	0		1

```
In [12]: #check the correlation to imputation
plt.figure(figsize=(10, 10))
sns.heatmap(numeric_df.corr(),annot=True)
```

Out[12]: <AxesSubplot:>



```
In [15]: sns.histplot(x='number_of_defaulted_accounts', data=df)
plt.title('Distribution of number_of_defaulted_accounts by Charge Off Status')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.show()
```



```
In [16]: df_default_0 = df[df['number_of_defaulted_accounts'] == 0]
df_default_1 = df[df['number_of_defaulted_accounts'] == 1]
df_default_2 = df[df['number_of_defaulted_accounts'] == 2]
```

```
In [17]: df_default_0['fico_score'].mode()
```

```
Out[17]: 0    650.0
1    698.0
Name: fico_score, dtype: float64
```

```
In [18]: df_default_1['fico_score'].mode()
```

```
Out[18]: 0    557.0
Name: fico_score, dtype: float64
```

```
In [19]: df_default_2['fico_score'].mode()
```

```
Out[19]: 0    484.0
1    488.0
Name: fico_score, dtype: float64
```

In [20]:

```
def impute_with_random_mode(df_column):

    modes = df_column.mode()
    if len(modes) > 1: # for multiple modes
        random_choices = np.random.choice(modes, size=df_column.isna().sum())
        df_column.loc[df_column.isna()] = random_choices
    else: # for only one mode
        df_column.fillna(modes[0], inplace=True)

# Imputing fico_score for each group
impute_with_random_mode(df_default_0['fico_score'])
impute_with_random_mode(df_default_1['fico_score'])
impute_with_random_mode(df_default_2['fico_score'])
```

C:\Users\THUSHAN\AppData\Local\Temp\ipykernel_2856\273746600.py:6: Setting WithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy (https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

```
df_column.loc[df_column.isna()] = random_choices
```

C:\Users\THUSHAN\AppData\Local\Temp\ipykernel_2856\273746600.py:8: Setting WithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy (https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

```
df_column.fillna(modes[0], inplace=True)
```

C:\Users\THUSHAN\AppData\Local\Temp\ipykernel_2856\273746600.py:6: Setting WithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy (https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

```
df_column.loc[df_column.isna()] = random_choices
```

In [21]: `df = pd.concat([df_default_0,df_default_1,df_default_2])`

```
In [22]: imputer = KNNImputer(n_neighbors=2)

# Apply KNN Imputation on the selected columns

df_imputed1= imputer.fit_transform(df[['number_of_delinquent_accounts']])

df_imputed2=imputer.fit_transform(df[['unusual_submission_pattern']])
df_imputed3=imputer.fit_transform(df[['avg_balance_last_12months']])

# Replace the original value into column with imputed values

df['number_of_delinquent_accounts'] = df_imputed1[:, 0].round()
df['unusual_submission_pattern'] = df_imputed2[:, 0].round()
df['avg_balance_last_12months'] = df_imputed3[:, 0]
```

```
In [23]: df.isnull().sum()
```

```
Out[23]: account_open_date      0
age      0
location  0
occupation  0
income_level  0
fico_score  0
delinquency_status  0
charge_off_status  0
number_of_credit_applications  0
debt_to_income_ratio  0
payment_methods_high_risk  0
max_balance  0
avg_balance_last_12months  0
number_of_delinquent_accounts  0
number_of_defaulted_accounts  0
earliest_credit_account  0
recent_trade_activity  0
new_accounts_opened_last_12months  0
multiple_applications_short_time_period  0
unusual_submission_pattern  0
applications_submitted_during_odd_hours  0
watchlist_blacklist_flag  0
public_records_flag  0
dtype: int64
```

In [24]: `df.dtypes`

```
Out[24]: account_open_date      datetime64[ns]
age                          int64
location                     object
occupation                   object
income_level                 int64
fico_score                   float64
delinquency_status          int64
charge_off_status           bool
number_of_credit_applications int64
debt_to_income_ratio        float64
payment_methods_high_risk    bool
max_balance                  float64
avg_balance_last_12months    float64
number_of_delinquent_accounts float64
number_of_defaulted_accounts int64
earliest_credit_account      datetime64[ns]
recent_trade_activity        datetime64[ns]
new_accounts_opened_last_12months int64
multiple_applications_short_time_period bool
unusual_submission_pattern    float64
applications_submitted_during_odd_hours bool
watchlist_blacklist_flag     bool
public_records_flag          bool
dtype: object
```

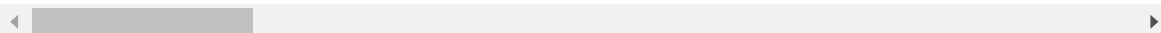
In [25]: `df['unusual_submission_pattern'] = df['unusual_submission_pattern'].astype(bool)`

In [26]: `df`

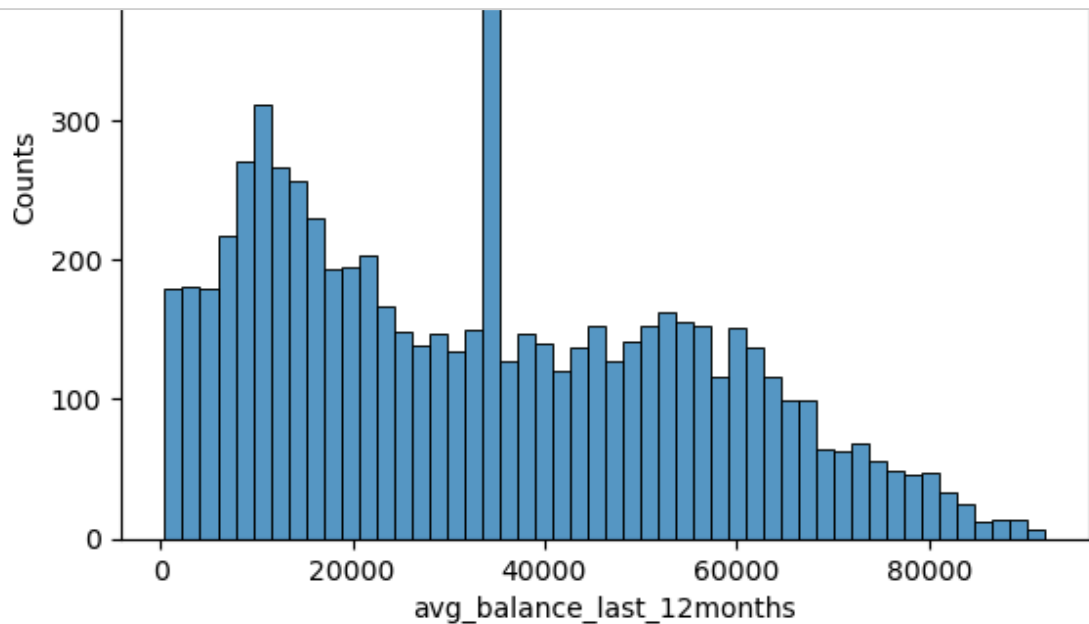
```
Out[26]:
```

	account_open_date	age	location	occupation	income_level	fico_score	delinquency_s
2	2020-07-12	46	Miami	Engineer	71936	566.0	
4	2024-07-27	60	Houston	Lawyer	8574	787.0	
5	2022-03-06	25	Chicago	Teacher	59349	688.0	
6	2023-10-26	38	Los Angeles	Banker	6925	740.0	
8	2022-09-10	36	Los Angeles	Other	6197	758.0	
...	...	...	...	...	...	...	...
6960	2021-01-17	65	San Francisco	Artist	10876	447.0	
6989	2024-09-15	29	Boston	Consultant	16347	469.0	
6990	2019-12-27	74	Chicago	Doctor	51012	580.0	
6995	2021-04-10	33	Boston	Consultant	14566	589.0	
6997	2024-06-20	68	Houston	Other	4501	321.0	

7000 rows × 23 columns



```
In [27]: plt.figure(figsize=(10, 5))
for i in range(len(missing_values_columns)):
    sns.histplot(df[missing_values_columns[i] ], bins=50)
    plt.title(f"Distribution of {missing_values_columns[i] } Data ")
    plt.xlabel(missing_values_columns[i])
    plt.ylabel('Counts')
    plt.show()
```



```
In [28]: df.duplicated().sum()
```

Out[28]: 0

```
In [29]: df
```

Out[29]:

	account_open_date	age	location	occupation	income_level	fico_score	delinquency_s
2	2020-07-12	46	Miami	Engineer	71936	566.0	
4	2024-07-27	60	Houston	Lawyer	8574	787.0	
5	2022-03-06	25	Chicago	Teacher	59349	688.0	
6	2023-10-26	38	Los Angeles	Banker	6925	740.0	
8	2022-09-10	36	Los Angeles	Other	6197	758.0	
...	...	...	...	...	...	...	
6960	2021-01-17	65	San Francisco	Artist	10876	447.0	
6989	2024-09-15	29	Boston	Consultant	16347	469.0	
6990	2019-12-27	74	Chicago	Doctor	51012	580.0	
6995	2021-04-10	33	Boston	Consultant	14566	589.0	
6997	2024-06-20	68	Houston	Other	4501	321.0	

7000 rows × 23 columns

```
In [30]: df['date_contradiction_flag'] = (  
    (df['earliest_credit_account'] > df['account_open_date']) |  
    (df['account_open_date'] > df['recent_trade_activity']) |  
    (df['earliest_credit_account'] > df['recent_trade_activity'])  
)
```

```
In [31]: df['date_contradiction_flag'].value_counts()
```

```
Out[31]: False    4570  
        True     2430  
        Name: date_contradiction_flag, dtype: int64
```

```
In [32]: contradiction_count = df['date_contradiction_flag'].sum()  
print(f"Number of rows with date contradictions: {contradiction_count}")
```

Number of rows with date contradictions: 2430

```
In [33]: df.loc[df['date_contradiction_flag'], 'earliest_credit_account'] = df[['earliest_credit_account', 'recent_trade_activity']]  
df.loc[df['date_contradiction_flag'], 'recent_trade_activity'] = df[['earliest_credit_account', 'recent_trade_activity']]
```

```
In [34]: df
```

Out[34]:

	account_open_date	age	location	occupation	income_level	fico_score	delinquency_s
2	2020-07-12	46	Miami	Engineer	71936	566.0	
4	2024-07-27	60	Houston	Lawyer	8574	787.0	
5	2022-03-06	25	Chicago	Teacher	59349	688.0	
6	2023-10-26	38	Los Angeles	Banker	6925	740.0	
8	2022-09-10	36	Los Angeles	Other	6197	758.0	
...	...	...	...	...	...	...	
6960	2021-01-17	65	San Francisco	Artist	10876	447.0	
6989	2024-09-15	29	Boston	Consultant	16347	469.0	
6990	2019-12-27	74	Chicago	Doctor	51012	580.0	
6995	2021-04-10	33	Boston	Consultant	14566	589.0	
6997	2024-06-20	68	Houston	Other	4501	321.0	

7000 rows × 24 columns

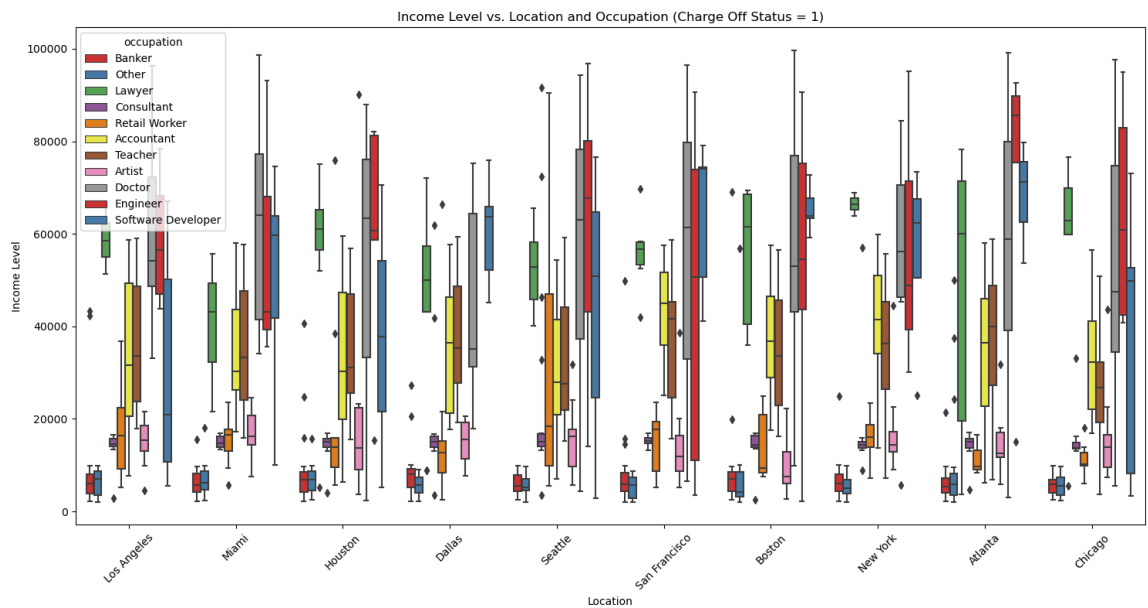
In [35]:

```

# Filter the DataFrame to include only rows where charge_off_status is 1
df_charge_off_1 = df[df['charge_off_status'] == 1]
plt.figure(figsize=(15, 8))

# Boxplot: income vs. Location and occupation, considering only charge_off_status = 1
sns.boxplot(x='location', y='income_level', data=df_charge_off_1, hue='occupation')
plt.title('Income Level vs. Location and Occupation (Charge Off Status = 1)')
plt.ylabel('Income Level')
plt.xlabel('Location')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

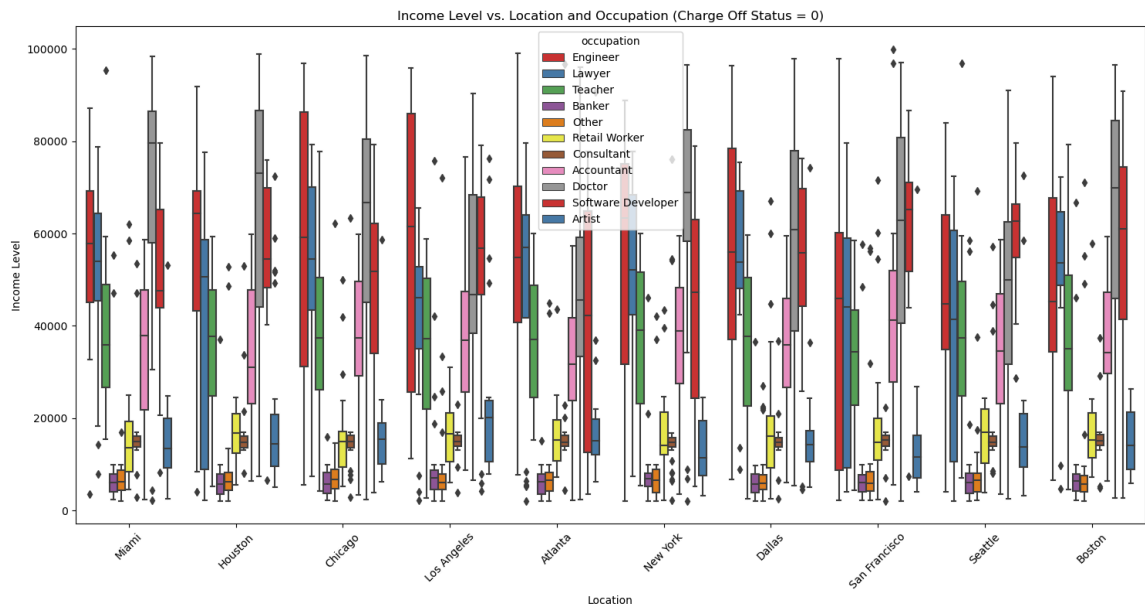
```



In [36]:

```
# Filter the DataFrame to include only rows where charge_off_status is 0
df_charge_off_0 = df[df['charge_off_status'] == 0]
plt.figure(figsize=(15, 8))

# Boxplot: income vs. Location and occupation, considering only charge_off_status = 0
sns.boxplot(x='location', y='income_level', data=df_charge_off_0, hue='occupation')
plt.title('Income Level vs. Location and Occupation (Charge Off Status = 0)')
plt.ylabel('Income Level')
plt.xlabel('Location')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

In [37]: `def remove_outliers_by_group(df, group_cols, value_col):`

```
    Q1 = df.groupby(group_cols)[value_col].transform(lambda x: x.quantile(0.25))
    Q3 = df.groupby(group_cols)[value_col].transform(lambda x: x.quantile(0.75))
    IQR = Q3 - Q1
```

```
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
```

```
    df = df[(df[value_col] >= lower_bound) & (df[value_col] <= upper_bound)]
    return df
```

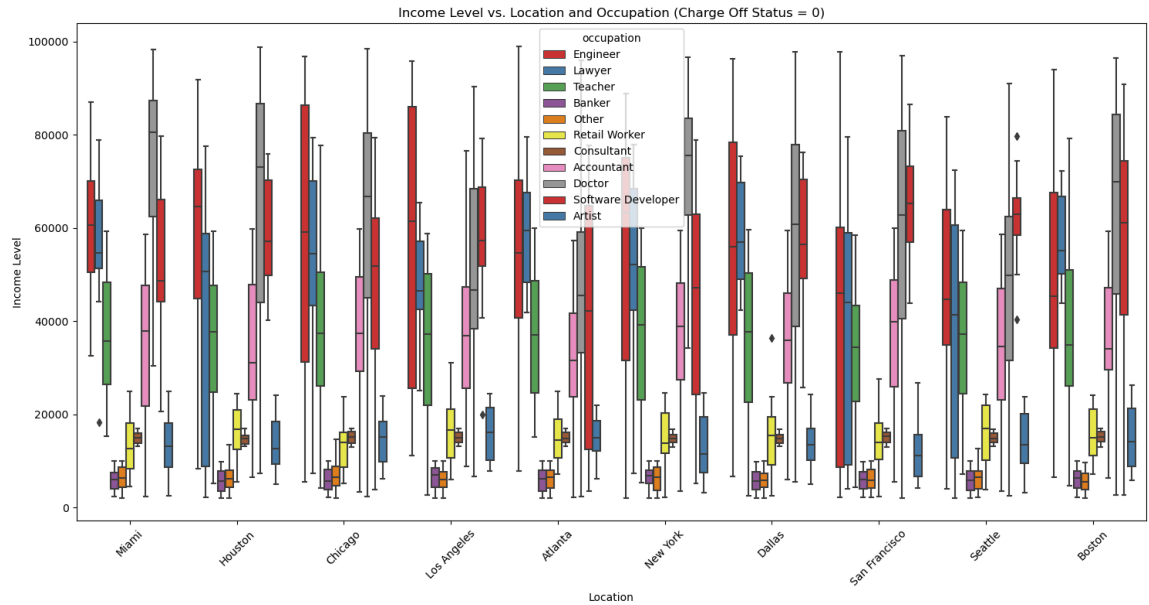
```
df_no_outliers_charge_off_0 = remove_outliers_by_group(df_charge_off_0, group_cols, 'income_level')
```

```
df_no_outliers_charge_off_1 = remove_outliers_by_group(df_charge_off_1, group_cols, 'income_level')
```

In [38]:

```
plt.figure(figsize=(15, 8))

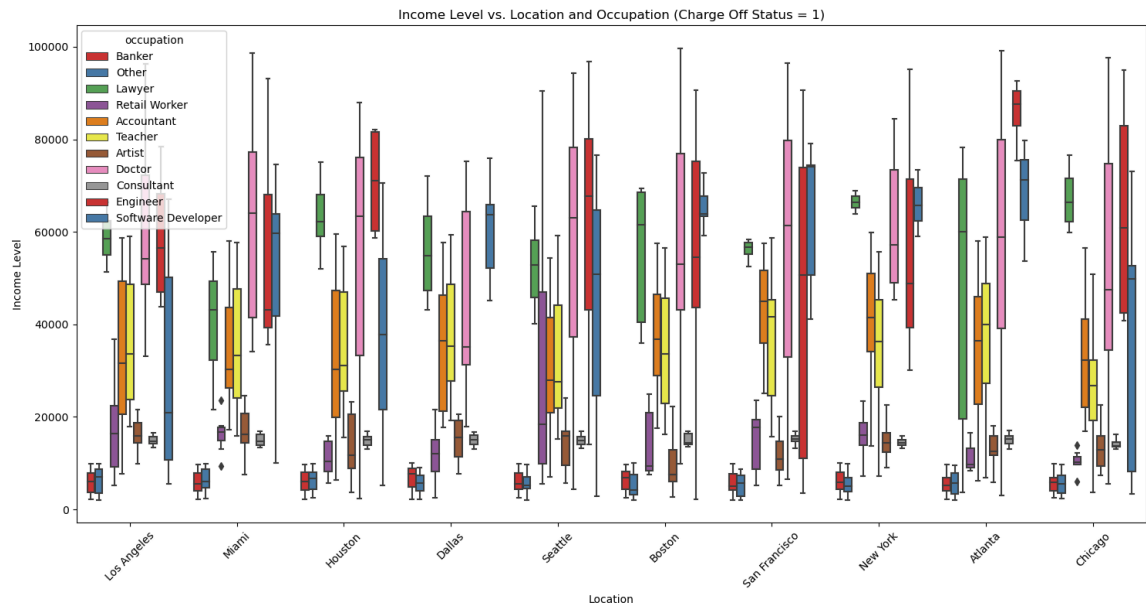
# Boxplot: income vs. Location and occupation, considering only charge_off_status = 0
sns.boxplot(x='location', y='income_level', data=df_no_outliers_charge_off_status_0)
plt.title('Income Level vs. Location and Occupation (Charge Off Status = 0)')
plt.ylabel('Income Level')
plt.xlabel('Location')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



In [39]:

```
plt.figure(figsize=(15, 8))

# Boxplot: income vs. Location and occupation, considering only charge_off_status = 1
sns.boxplot(x='location', y='income_level', data=df_no_outliers_charge_off_status_1)
plt.title('Income Level vs. Location and Occupation (Charge Off Status = 1)')
plt.ylabel('Income Level')
plt.xlabel('Location')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

In [40]: `df = pd.concat([df_no_outliers_charge_off_0 , df_no_outliers_charge_off_1])`In [41]: `df`

Out[41]:

	account_open_date	age	location	occupation	income_level	fico_score	delinquency_s
2	2020-07-12	46	Miami	Engineer	71936	566.0	
4	2024-07-27	60	Houston	Lawyer	8574	787.0	
5	2022-03-06	25	Chicago	Teacher	59349	688.0	
6	2023-10-26	38	Los Angeles	Banker	6925	740.0	
8	2022-09-10	36	Los Angeles	Other	6197	758.0	
...	...	...	...	...	...	...	
6960	2021-01-17	65	San Francisco	Artist	10876	447.0	
6989	2024-09-15	29	Boston	Consultant	16347	469.0	
6990	2019-12-27	74	Chicago	Doctor	51012	580.0	
6995	2021-04-10	33	Boston	Consultant	14566	589.0	
6997	2024-06-20	68	Houston	Other	4501	321.0	

6784 rows × 24 columns

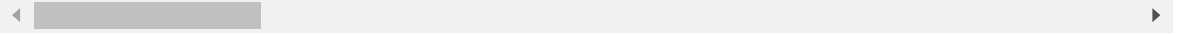
In [42]:

```
outliers_age = df[((df['age'] - df['age'].mean()) / df['age'].std()).abs() > 3]  
outliers_age
```

Out[42]:

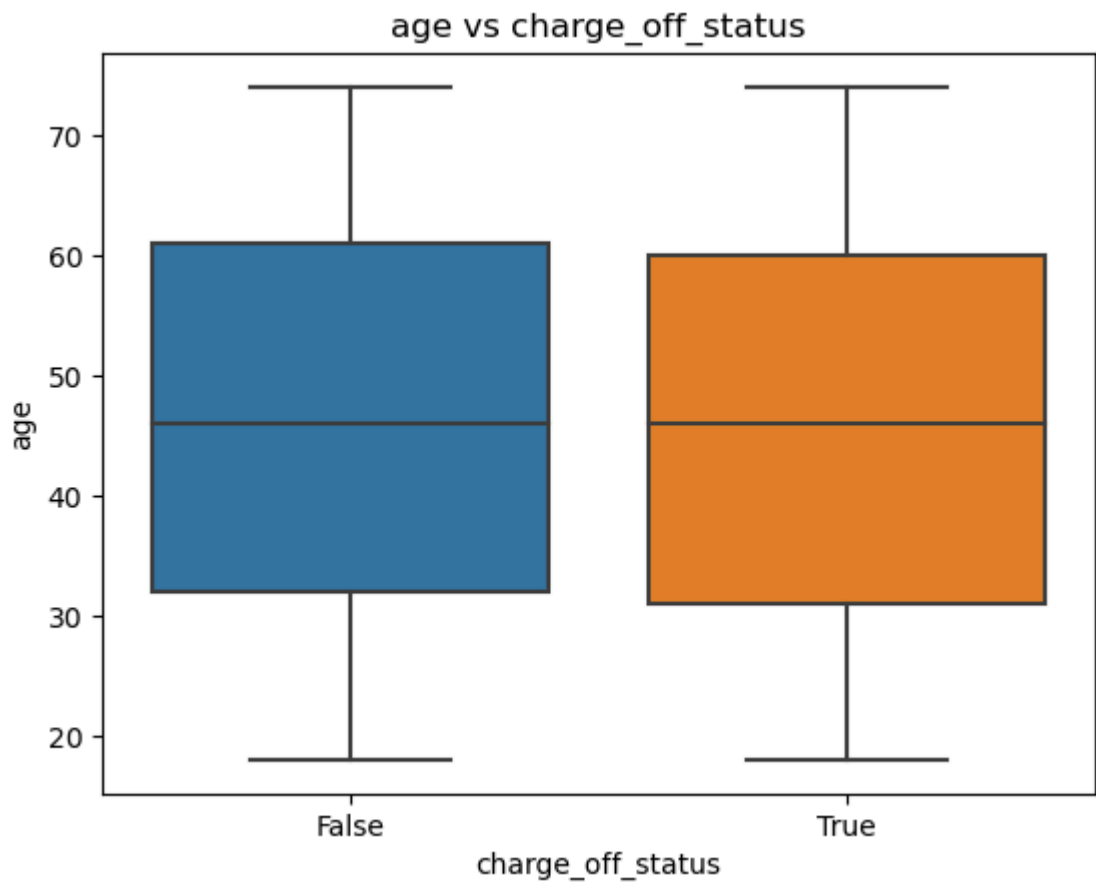
account_open_date	age	location	occupation	income_level	fico_score	delinquency_status
-------------------	-----	----------	------------	--------------	------------	--------------------

0 rows × 24 columns



In [43]:

```
sns.boxplot(x=df['charge_off_status'], y=df['age'])  
plt.title('age vs charge_off_status')  
plt.ylabel('age')  
plt.xlabel('charge_off_status')  
plt.show()
```



```
In [44]: outliers_fico_score = df[(df['fico_score'] < 300) | (df['fico_score'] > 850)]  
outliers_fico_score
```

Out[44]:

	account_open_date	age	location	occupation	income_level	fico_score	delinquency_3
862	2024-02-19	46	Miami	Banker	3202	938.0	
908	2021-03-25	48	Seattle	Accountant	39792	278.0	
920	2024-05-03	34	Boston	Other	7897	906.0	
1288	2021-06-29	40	Boston	Accountant	45827	931.0	
1989	2023-11-06	23	Seattle	Teacher	45887	940.0	
2050	2022-05-27	30	Houston	Other	7568	243.0	
2158	2024-09-28	36	Boston	Teacher	27337	916.0	
2287	2024-05-26	26	Los Angeles	Teacher	24534	174.0	
2893	2021-07-14	62	Seattle	Accountant	24097	234.0	
3549	2020-03-09	48	Atlanta	Other	5151	908.0	
3978	2023-10-24	52	New York	Accountant	22013	903.0	
4271	2022-04-26	41	Seattle	Banker	5753	241.0	
4294	2020-07-07	52	Seattle	Lawyer	64826	152.0	
4466	2023-05-05	74	Seattle	Accountant	12278	913.0	
4723	2022-04-13	24	Miami	Banker	4270	255.0	
4867	2023-09-21	43	Boston	Lawyer	72255	196.0	
5296	2024-08-13	66	Houston	Consultant	13826	183.0	
5524	2020-05-31	58	New York	Engineer	67105	914.0	
5641	2019-10-31	52	New York	Teacher	42383	940.0	
6018	2024-05-21	28	Chicago	Accountant	31640	936.0	
6376	2024-05-21	70	Miami	Accountant	19168	900.0	
6549	2024-01-20	36	New York	Banker	7661	902.0	
6673	2024-05-30	61	Boston	Other	8028	217.0	
4299	2024-06-15	26	San Francisco	Doctor	49055	901.0	
5202	2024-01-18	38	Dallas	Other	5610	909.0	
6729	2020-07-23	51	Miami	Teacher	56174	279.0	
1110	2024-03-18	43	Boston	Artist	5879	287.0	
2779	2022-05-11	52	Los Angeles	Teacher	21634	920.0	
3419	2024-05-31	72	Houston	Teacher	56837	922.0	
5640	2020-07-02	36	Miami	Teacher	47439	913.0	
6567	2020-08-18	49	Atlanta	Teacher	42774	154.0	
6591	2021-05-08	55	Atlanta	Banker	2335	200.0	
161	2024-08-11	26	Houston	Other	6959	923.0	
2219	2023-06-17	34	Miami	Doctor	78377	913.0	
4087	2022-08-22	56	Seattle	Teacher	25870	943.0	
6681	2023-03-28	52	Seattle	Other	2281	918.0	

	account_open_date	age	location	occupation	income_level	fico_score	delinquency_s
131	2022-02-26	24	Miami	Software Developer	9972	920.0	
2671	2023-09-02	40	New York	Banker	3133	205.0	
3921	2023-06-28	64	Miami	Other	8846	924.0	
4157	2023-05-16	68	Boston	Banker	3732	915.0	
4811	2023-12-04	33	Chicago	Accountant	22595	902.0	

41 rows × 24 columns

```
In [45]: # Remove fico_score outliers from the dataset
df = df[(df['fico_score'] >= 300) & (df['fico_score'] <= 850)]
df.reset_index(drop=True, inplace=True)

df
```

Out[45]:

	account_open_date	age	location	occupation	income_level	fico_score	delinquency_s
0	2020-07-12	46	Miami	Engineer	71936	566.0	
1	2024-07-27	60	Houston	Lawyer	8574	787.0	
2	2022-03-06	25	Chicago	Teacher	59349	688.0	
3	2023-10-26	38	Los Angeles	Banker	6925	740.0	
4	2022-09-10	36	Los Angeles	Other	6197	758.0	
...	...	...	...	...	...	...	...
6738	2021-01-17	65	San Francisco	Artist	10876	447.0	
6739	2024-09-15	29	Boston	Consultant	16347	469.0	
6740	2019-12-27	74	Chicago	Doctor	51012	580.0	
6741	2021-04-10	33	Boston	Consultant	14566	589.0	
6742	2024-06-20	68	Houston	Other	4501	321.0	

6743 rows × 24 columns

```
In [46]: outliers_fico_score = df[(df['fico_score'] < 300) | (df['fico_score'] > 850)]
outliers_fico_score
```

Out[46]:

	account_open_date	age	location	occupation	income_level	fico_score	delinquency_status
--	-------------------	-----	----------	------------	--------------	------------	--------------------

0 rows × 24 columns

In [47]:

```
outliers_delinquency_status = df[((df['delinquency_status'] - df['delinquency_status']) > 3) | (df['delinquency_status'] - df['delinquency_status'] < -3)]
outliers_delinquency_status
```

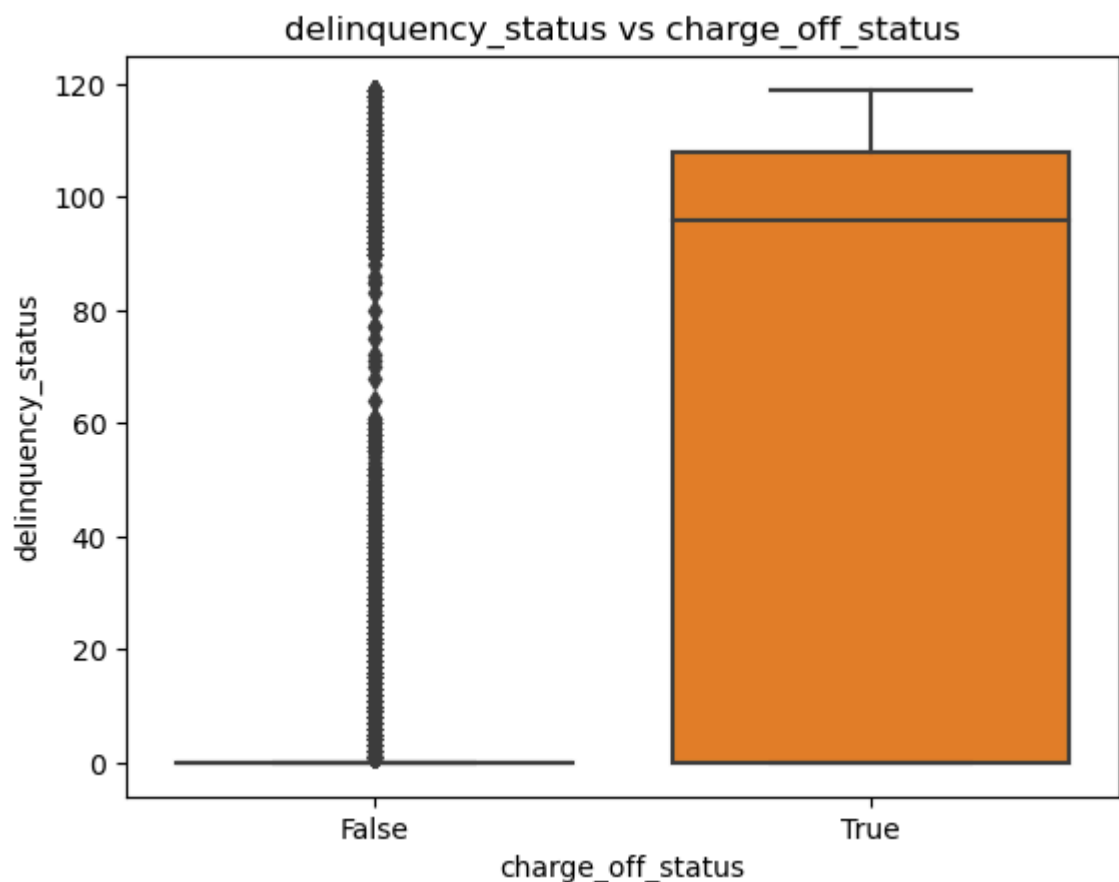
Out[47]:

```
account_open_date  age  location  occupation  income_level  fico_score  delinquency_status
```

```
0 rows × 24 columns
```

In [48]:

```
sns.boxplot(x=df['charge_off_status'], y=df['delinquency_status'])
plt.title('delinquency_status vs charge_off_status')
plt.ylabel('delinquency_status')
plt.xlabel('charge_off_status')
plt.show()
```



In [49]:

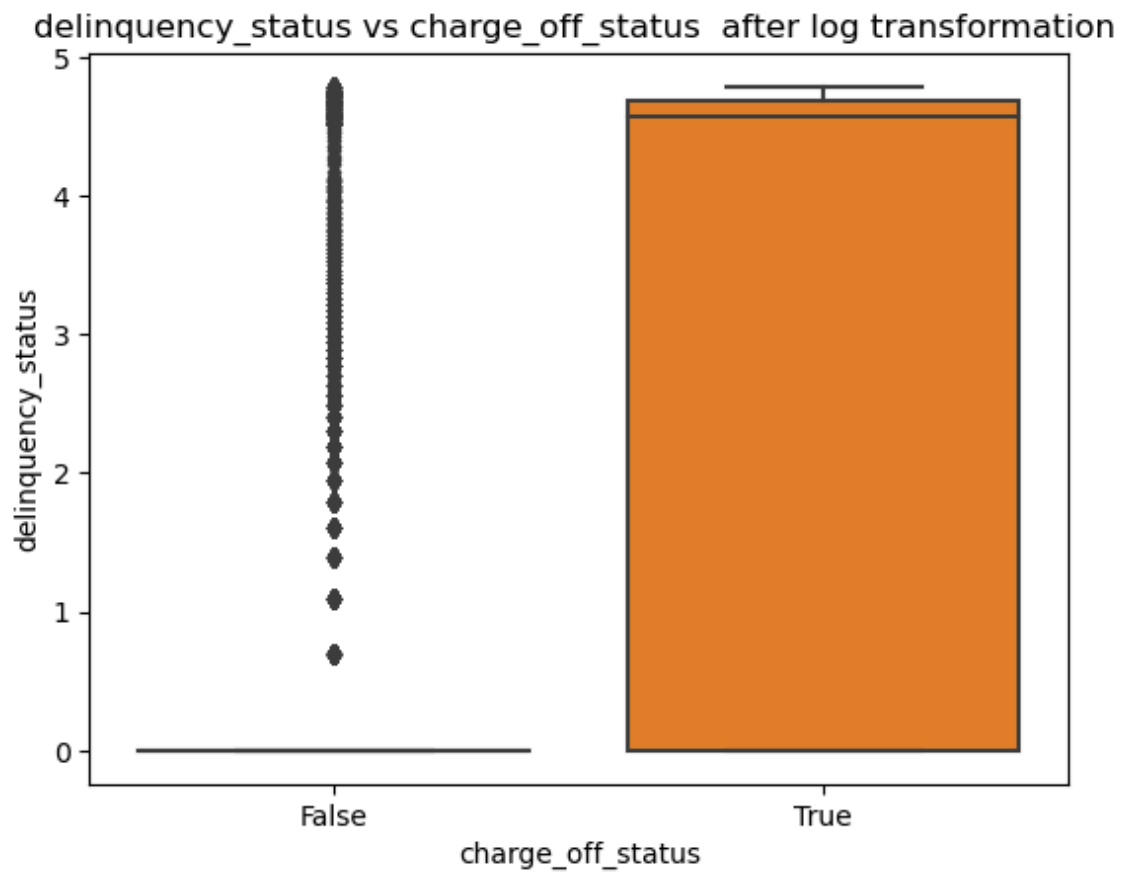
```
# Apply a log transformation
df['delinquency_status_log'] = np.log1p(df['delinquency_status'])
```

C:\Users\THUSHAN\AppData\Local\Temp\ipykernel_2856\3838396041.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy (https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

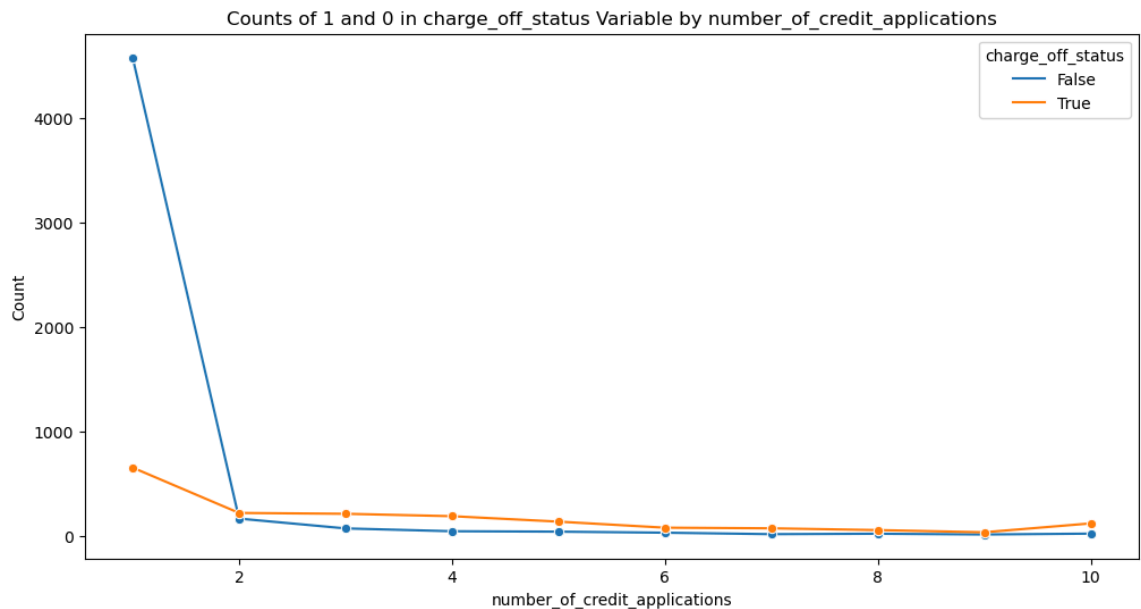
```
df['delinquency_status_log'] = np.log1p(df['delinquency_status'])
```

```
In [50]: sns.boxplot(x=df['charge_off_status'], y=df['delinquency_status_log'])  
plt.title('delinquency_status vs charge_off_status after log transformation')  
plt.ylabel('delinquency_status')  
plt.xlabel('charge_off_status')  
plt.show()
```

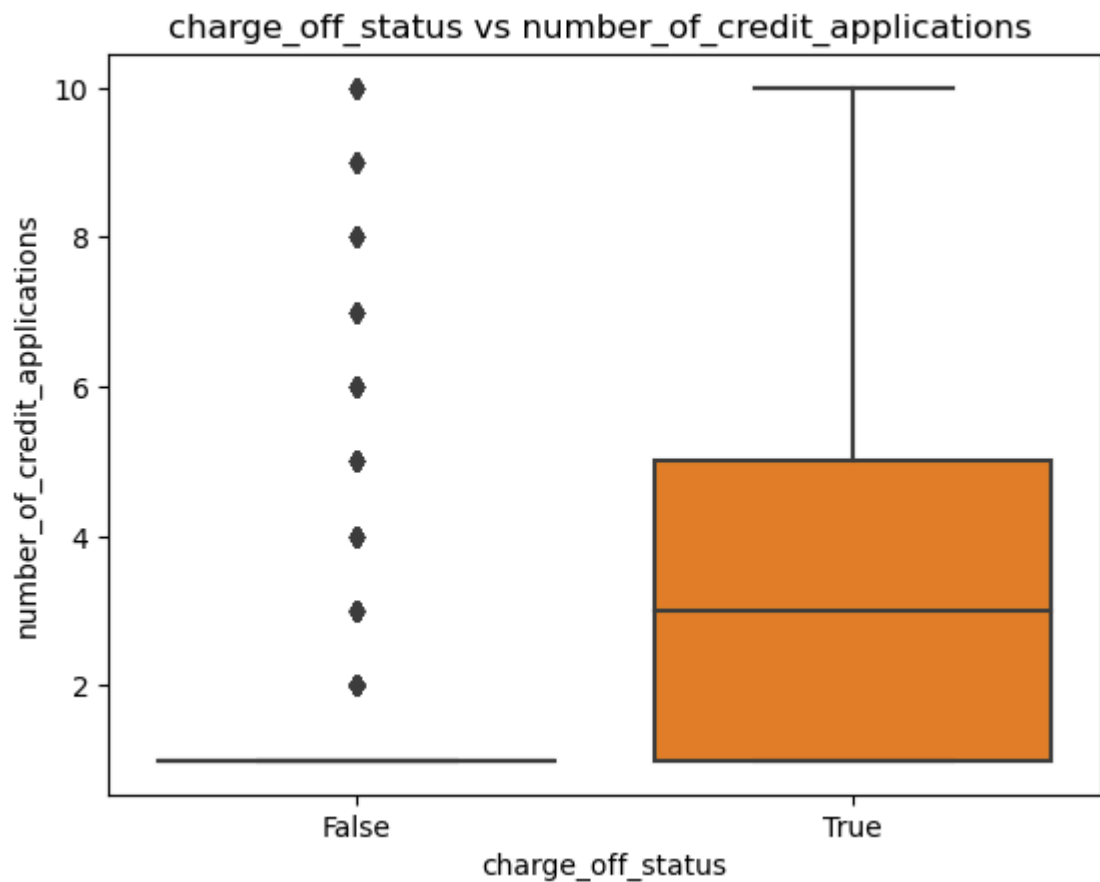


```
In [51]: # Group by date and accept, then count occurrences
grouped_df = df.groupby(['number_of_credit_applications', 'charge_off_status'])

# Plot
plt.figure(figsize=(12, 6))
sns.lineplot(data=grouped_df, x='number_of_credit_applications', y='count',
plt.title('Counts of 1 and 0 in charge_off_status Variable by number_of_cred
plt.xlabel('number_of_credit_applications')
plt.ylabel('Count')
plt.legend(title='charge_off_status')
plt.show()
```



```
In [52]: sns.boxplot(x=df['charge_off_status'], y=df['number_of_credit_applications'])  
plt.title('charge_off_status vs number_of_credit_applications ')  
plt.xlabel('charge_off_status')  
plt.ylabel('number_of_credit_applications')  
plt.show()
```

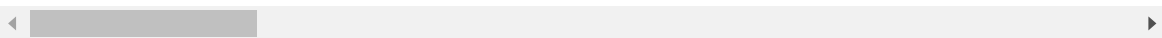


In [53]: *# see how many outliers in charge_off_status = 0 & number_of_credit_applications*
 filtered_charge_off_status_outliers = df[(df['charge_off_status'] == 0) & (df['number_of_credit_applications'] > 10)]
 filtered_charge_off_status_outliers

Out[53]:

	account_open_date	age	location	occupation	income_level	fico_score	delinquency_score
11	2021-04-04	41	Dallas	Software Developer	72993	672.0	
27	2024-03-24	31	Houston	Other	5598	650.0	
30	2021-11-12	45	Los Angeles	Other	5182	680.0	
36	2021-01-13	19	Dallas	Other	7781	508.0	
74	2023-07-16	18	Chicago	Banker	3749	652.0	
...	...	...	...	...	...	...	...
4979	2021-11-25	39	Miami	Other	9513	629.0	
4980	2021-07-02	34	San Francisco	Banker	9371	545.0	
4981	2021-10-13	22	San Francisco	Teacher	53408	622.0	
4983	2020-02-05	46	Houston	Retail Worker	22226	311.0	
4986	2019-11-25	30	Boston	Teacher	34390	459.0	

411 rows × 25 columns



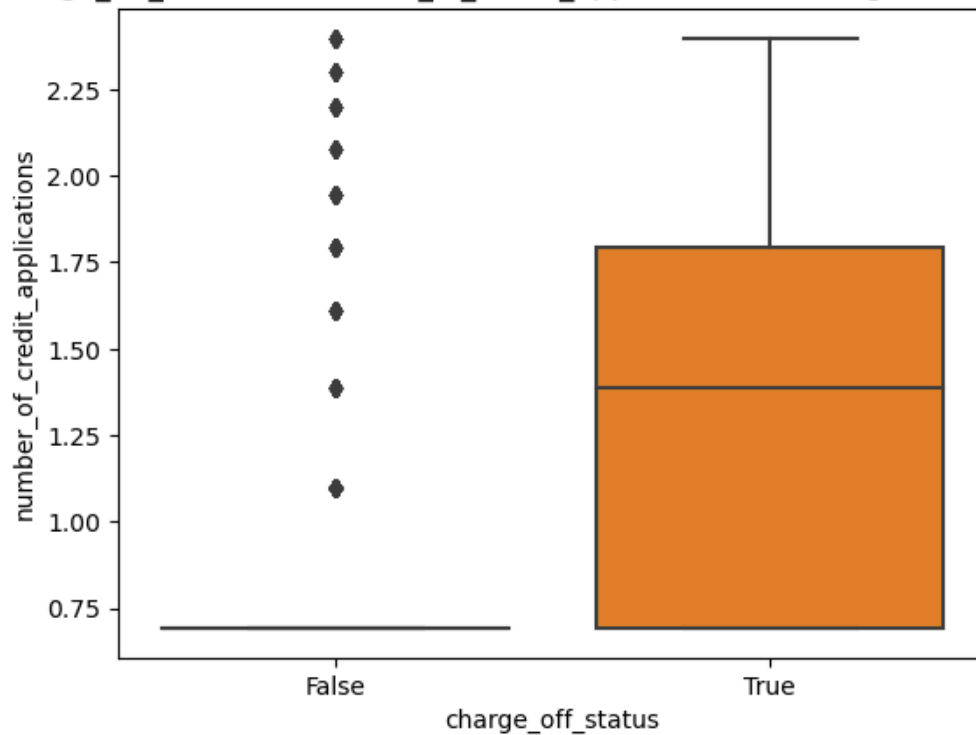
In [54]: *# Apply a log transformation*
 df['number_of_credit_applications_log'] = np.log1p(df['number_of_credit_applications'])

C:\Users\THUSHAN\AppData\Local\Temp\ipykernel_2856\286846169.py:2: SettingWithCopyWarning:
 A value is trying to be set on a copy of a slice from a DataFrame.
 Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy (https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)
 df['number_of_credit_applications_log'] = np.log1p(df['number_of_credit_applications'])

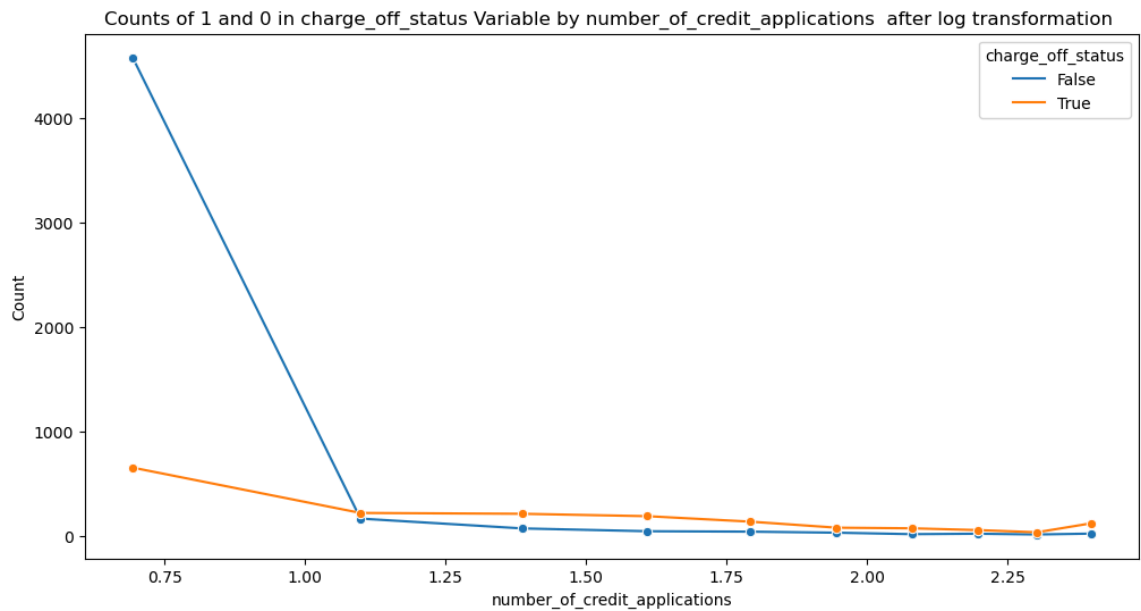
```
In [55]: sns.boxplot(x=df['charge_off_status'], y=df['number_of_credit_applications_'],  
plt.title('charge_off_status vs number_of_credit_applications after log tran  
plt.xlabel('charge_off_status')  
plt.ylabel('number_of_credit_applications')  
plt.show()
```

charge_off_status vs number_of_credit_applications after log transformation



```
In [56]: # Group by date and accept, then count occurrences
grouped_df = df.groupby(['number_of_credit_applications_log', 'charge_off_status'])

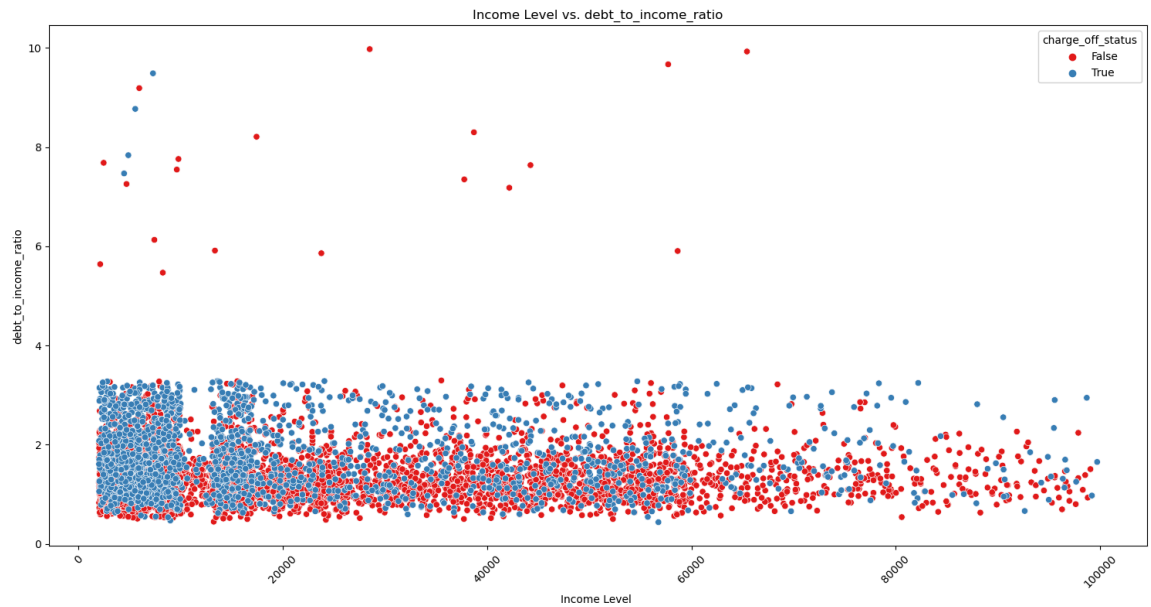
plt.figure(figsize=(12, 6))
sns.lineplot(data=grouped_df, x='number_of_credit_applications_log', y='count')
plt.title('Counts of 1 and 0 in charge_off_status Variable by number_of_credit_applications')
plt.xlabel('number_of_credit_applications')
plt.ylabel('Count')
plt.legend(title='charge_off_status')
plt.show()
```



```
In [57]: # Boxplot: income vs. Location and occupation
plt.figure(figsize=(15, 8))

sns.scatterplot(x='income_level', y='debt_to_income_ratio', data=df, hue='charge_off_status')

plt.title('Income Level vs. debt_to_income_ratio')
plt.xlabel('Income Level ')
plt.ylabel('debt_to_income_ratio')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



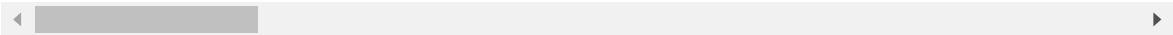
```
In [58]: # Remove outliers from the dataset
df = df[(df['debt_to_income_ratio'] <= 4) ]
df.reset_index(drop=True, inplace=True)

df
```

Out[58]:

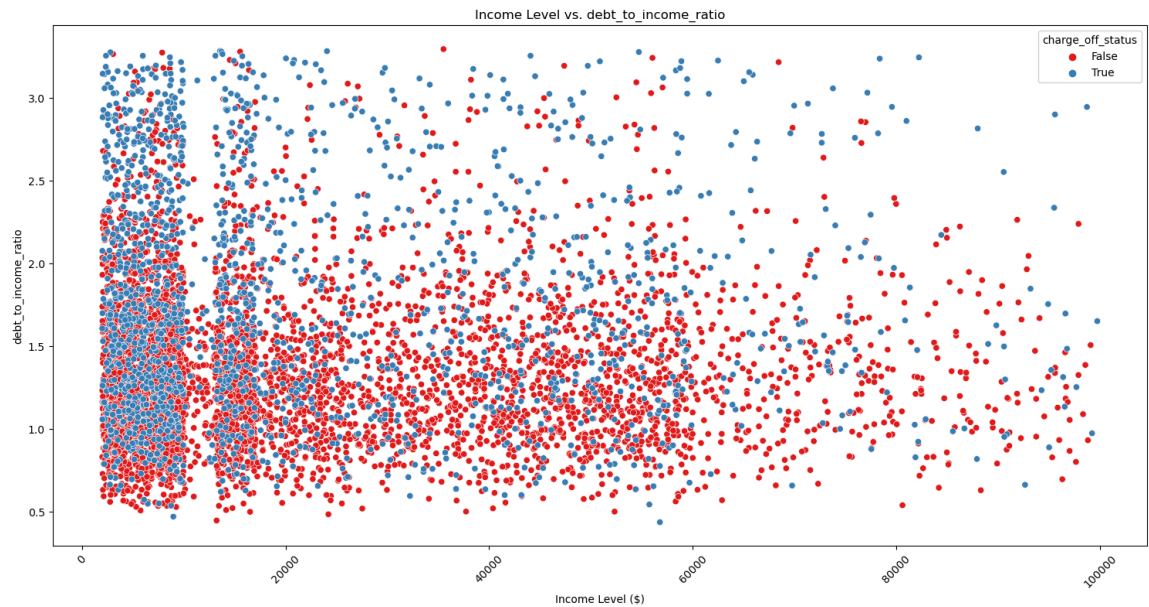
	account_open_date	age	location	occupation	income_level	fico_score	delinquency_
0	2020-07-12	46	Miami	Engineer	71936	566.0	
1	2024-07-27	60	Houston	Lawyer	8574	787.0	
2	2022-03-06	25	Chicago	Teacher	59349	688.0	
3	2023-10-26	38	Los Angeles	Banker	6925	740.0	
4	2022-09-10	36	Los Angeles	Other	6197	758.0	
...	...	...	...	...	...	...	
6715	2021-01-17	65	San Francisco	Artist	10876	447.0	
6716	2024-09-15	29	Boston	Consultant	16347	469.0	
6717	2019-12-27	74	Chicago	Doctor	51012	580.0	
6718	2021-04-10	33	Boston	Consultant	14566	589.0	
6719	2024-06-20	68	Houston	Other	4501	321.0	

6720 rows × 26 columns



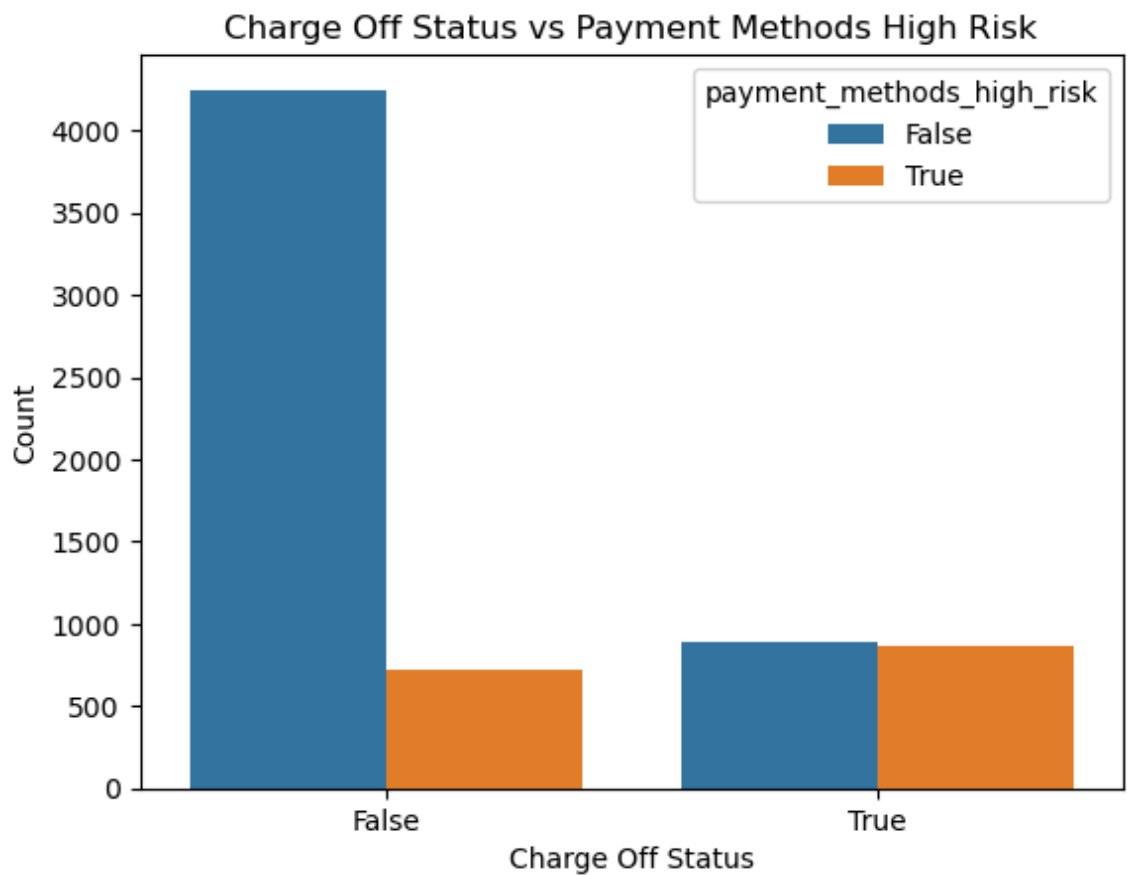
```
In [59]: # Boxplot: income vs. Location and occupation
plt.figure(figsize=(15, 8))
sns.scatterplot(x='income_level', y='debt_to_income_ratio', data=df, hue='charge_off_status')

plt.title('Income Level vs. debt_to_income_ratio')
plt.xlabel('Income Level ($)')
plt.ylabel('debt_to_income_ratio')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



In [60]:

```
sns.countplot(x='charge_off_status', hue='payment_methods_high_risk', data=c  
plt.title('Charge Off Status vs Payment Methods High Risk')  
plt.xlabel('Charge Off Status')  
plt.ylabel('Count')  
plt.show()
```

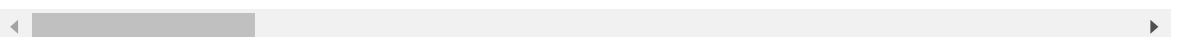


```
In [61]: outliers_max_balance = df[((df['max_balance'] - df['max_balance'].mean()) /
outliers_max_balance
```

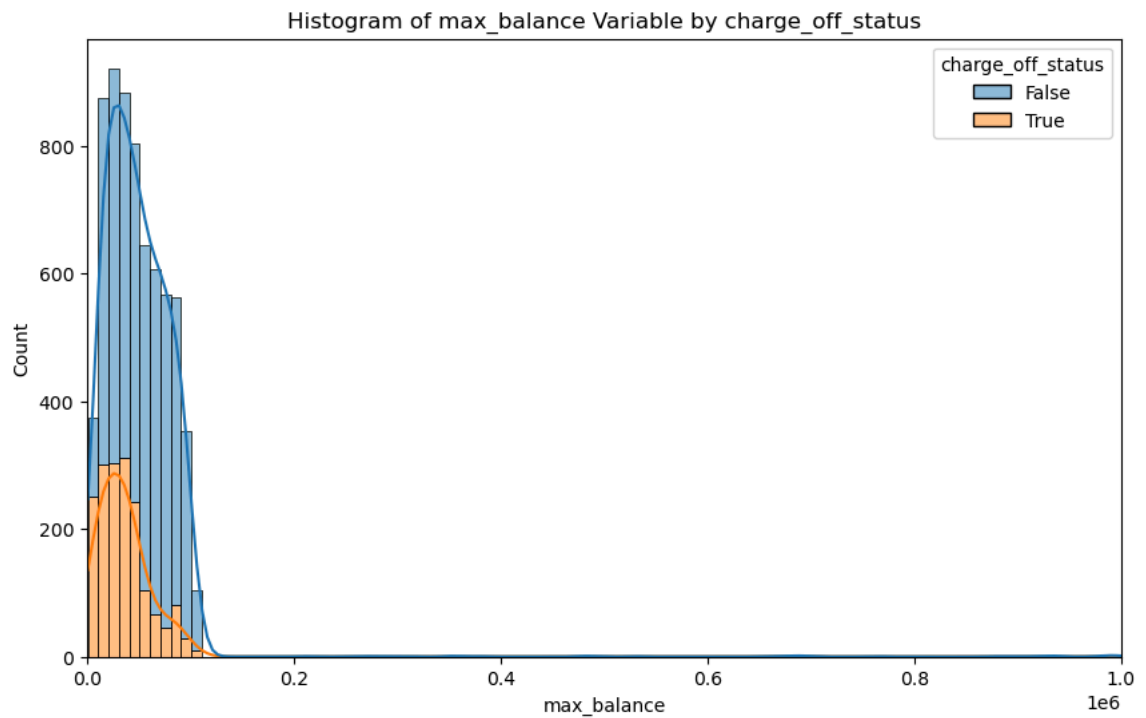
Out[61]:

	account_open_date	age	location	occupation	income_level	fico_score	delinquency_s
69	2019-11-24	54	Chicago	Teacher	44367	678.0	
112	2021-08-29	58	Atlanta	Consultant	15103	777.0	
221	2021-07-20	35	San Francisco	Banker	5669	626.0	
828	2020-02-04	33	Atlanta	Software Developer	44512	506.0	
850	2023-06-06	40	Dallas	Doctor	36557	652.0	
929	2024-07-26	70	Los Angeles	Banker	7532	834.0	
948	2024-08-05	51	Boston	Other	2218	791.0	
1628	2023-04-18	50	Seattle	Artist	12167	733.0	
1682	2021-07-14	40	Houston	Other	8054	516.0	
2949	2023-09-27	58	Atlanta	Doctor	82260	713.0	
3010	2022-04-17	24	Boston	Teacher	16769	748.0	
3276	2023-07-27	49	Los Angeles	Teacher	34366	779.0	
3637	2021-06-19	22	New York	Accountant	54643	713.0	
3753	2022-05-14	62	Dallas	Teacher	26838	810.0	
3785	2021-08-13	59	Los Angeles	Other	4241	653.0	
3880	2022-10-05	23	Boston	Accountant	46852	465.0	
3882	2024-04-25	43	Atlanta	Banker	7451	587.0	
4143	2024-05-19	71	Dallas	Software Developer	25819	511.0	
4521	2022-09-15	32	Boston	Accountant	53358	452.0	
4703	2022-05-02	22	Miami	Teacher	52207	328.0	
5269	2023-09-17	21	San Francisco	Accountant	57542	734.0	
5466	2024-03-27	66	Atlanta	Teacher	26110	521.0	
5554	2024-08-05	60	Dallas	Software Developer	45081	322.0	
5859	2022-09-08	28	Seattle	Software Developer	22652	571.0	
6159	2022-11-11	27	Boston	Other	2015	364.0	
6259	2023-07-16	42	New York	Banker	2112	521.0	
6713	2024-09-25	40	Los Angeles	Banker	3108	391.0	

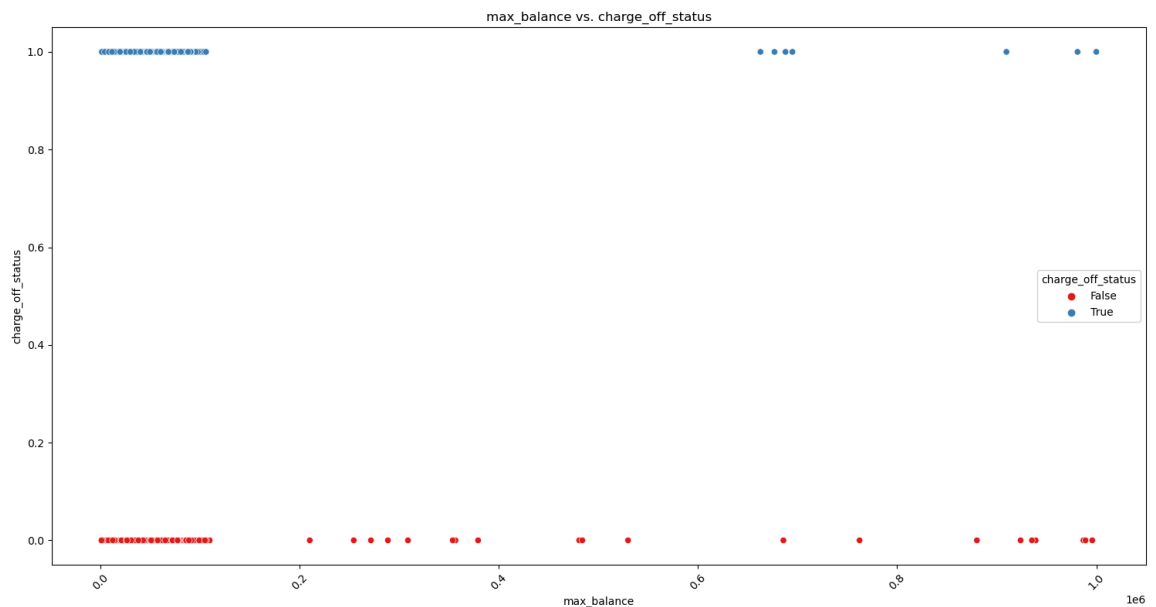
27 rows × 26 columns



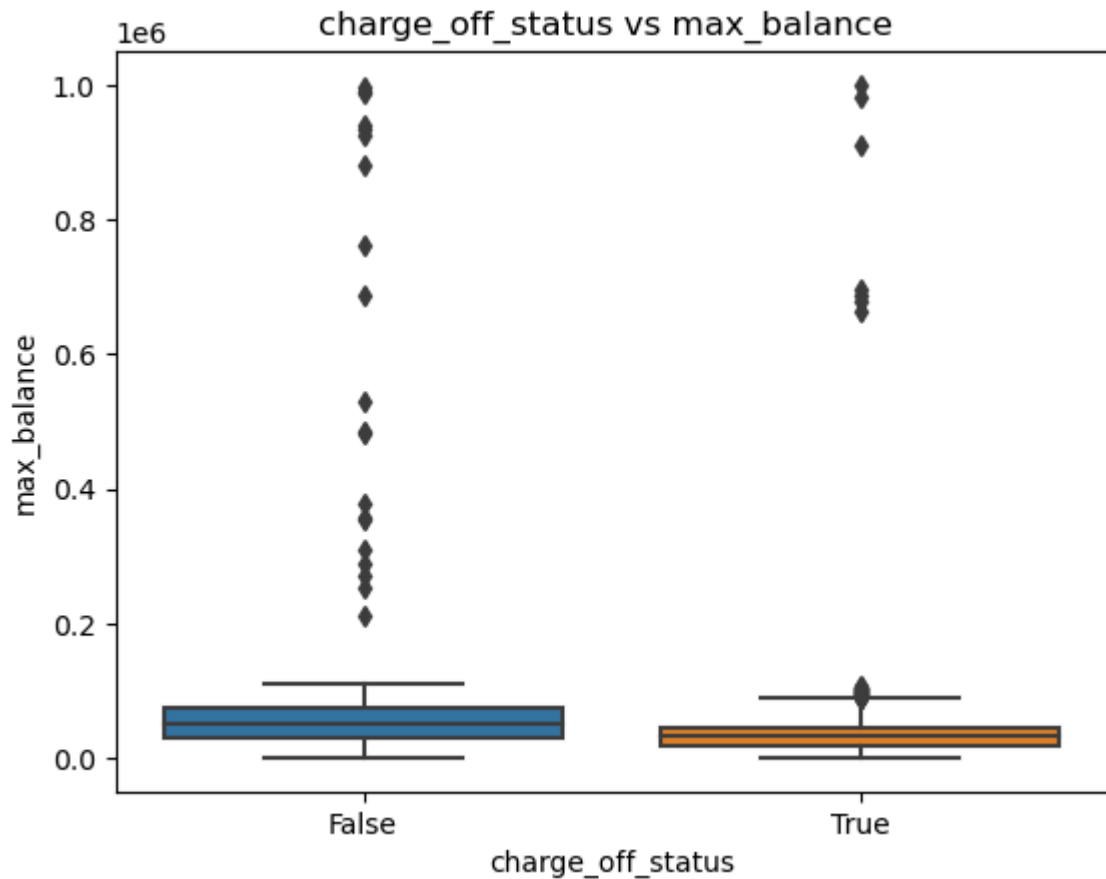
```
In [62]: plt.figure(figsize=(10, 6))
sns.histplot(data=df, x='max_balance', hue='charge_off_status', multiple='stack')
plt.xlabel('max_balance')
plt.xlim(0, 1000000)
plt.title('Histogram of max_balance Variable by charge_off_status')
plt.show()
```



```
In [63]: # Boxplot: income vs. Location and occupation
plt.figure(figsize=(15, 8))
sns.scatterplot(x='max_balance', y='charge_off_status', data=df, hue='charge_off_status')
plt.title('max_balance vs. charge_off_status')
plt.xlabel('max_balance')
plt.ylabel('charge_off_status')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



```
In [64]: sns.boxplot(x=df['charge_off_status'], y=df['max_balance'])
plt.title('charge_off_status vs max_balance ')
plt.xlabel('charge_off_status')
plt.show()
```



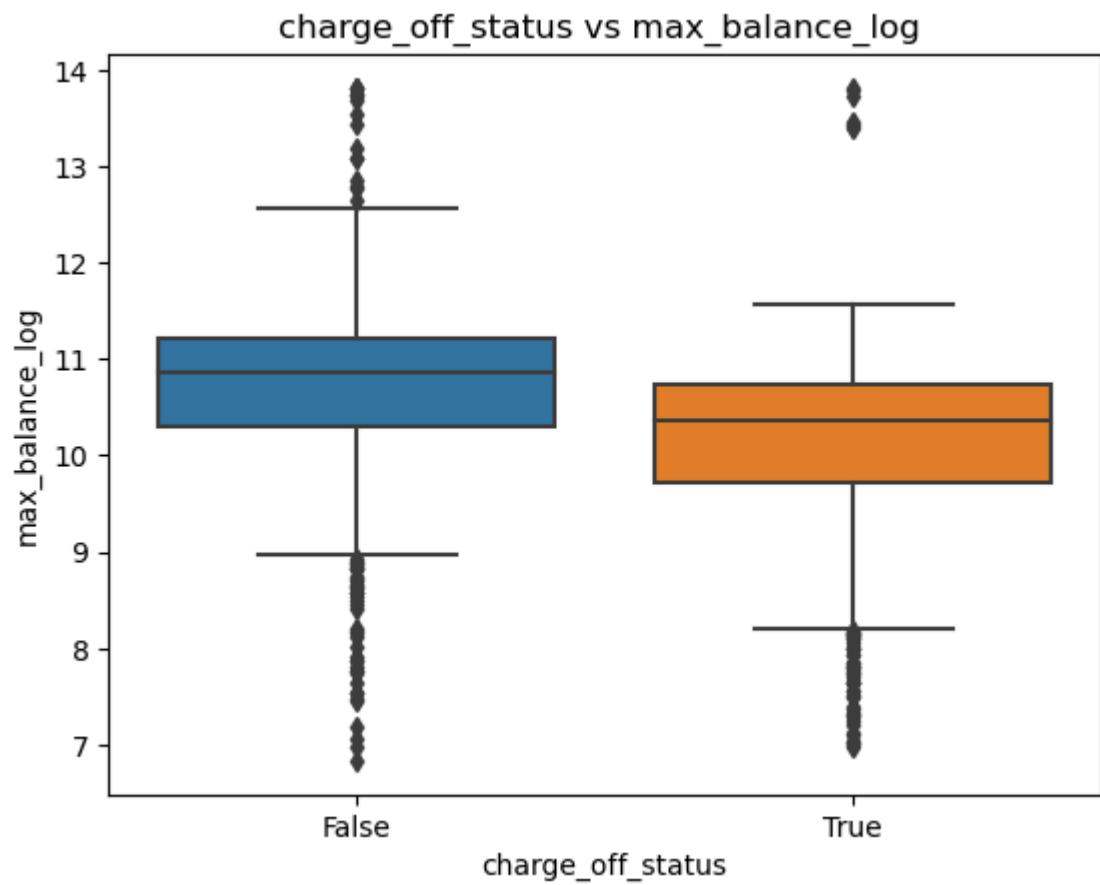
```
In [65]: # Apply a log transformation
df['max_balance_log'] = np.log1p(df['max_balance'])
```

C:\Users\THUSHAN\AppData\Local\Temp\ipykernel_2856\2433158796.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

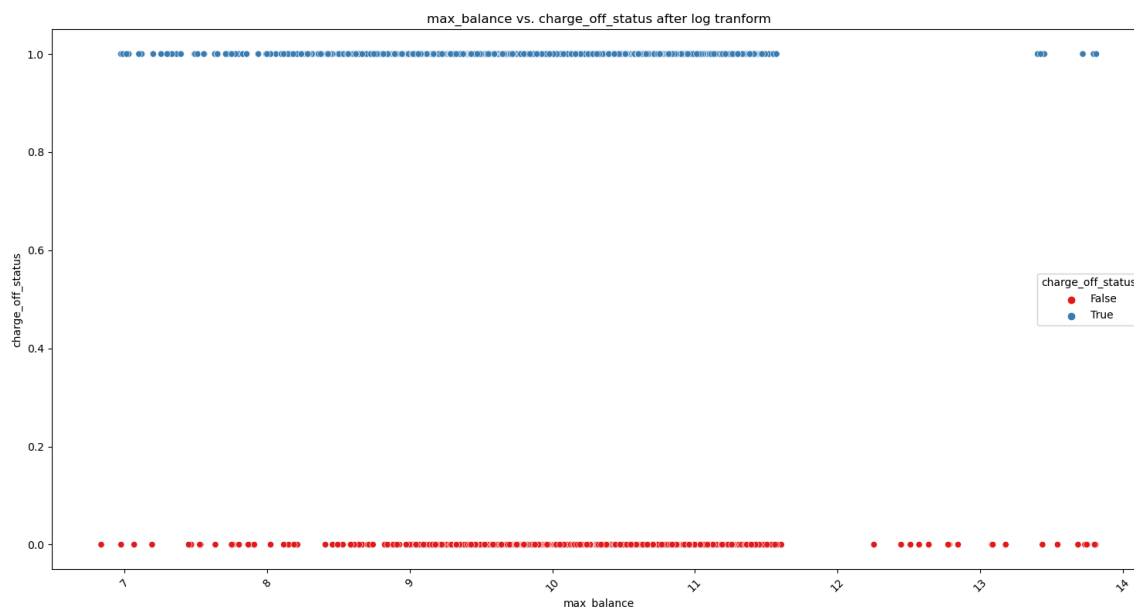
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy (https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

```
df['max_balance_log'] = np.log1p(df['max_balance'])
```

```
In [66]: sns.boxplot(x=df['charge_off_status'], y=df['max_balance_log'])  
plt.title('charge_off_status vs max_balance_log ')  
plt.xlabel('charge_off_status')  
plt.show()
```



```
In [67]: # Boxplot: income vs. Location and occupation
plt.figure(figsize=(15, 8))
sns.scatterplot(x='max_balance_log', y='charge_off_status', data=df, hue='charge_off_status')
plt.title('max_balance vs. charge_off_status after log tranform')
plt.xlabel('max_balance')
plt.ylabel('charge_off_status')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



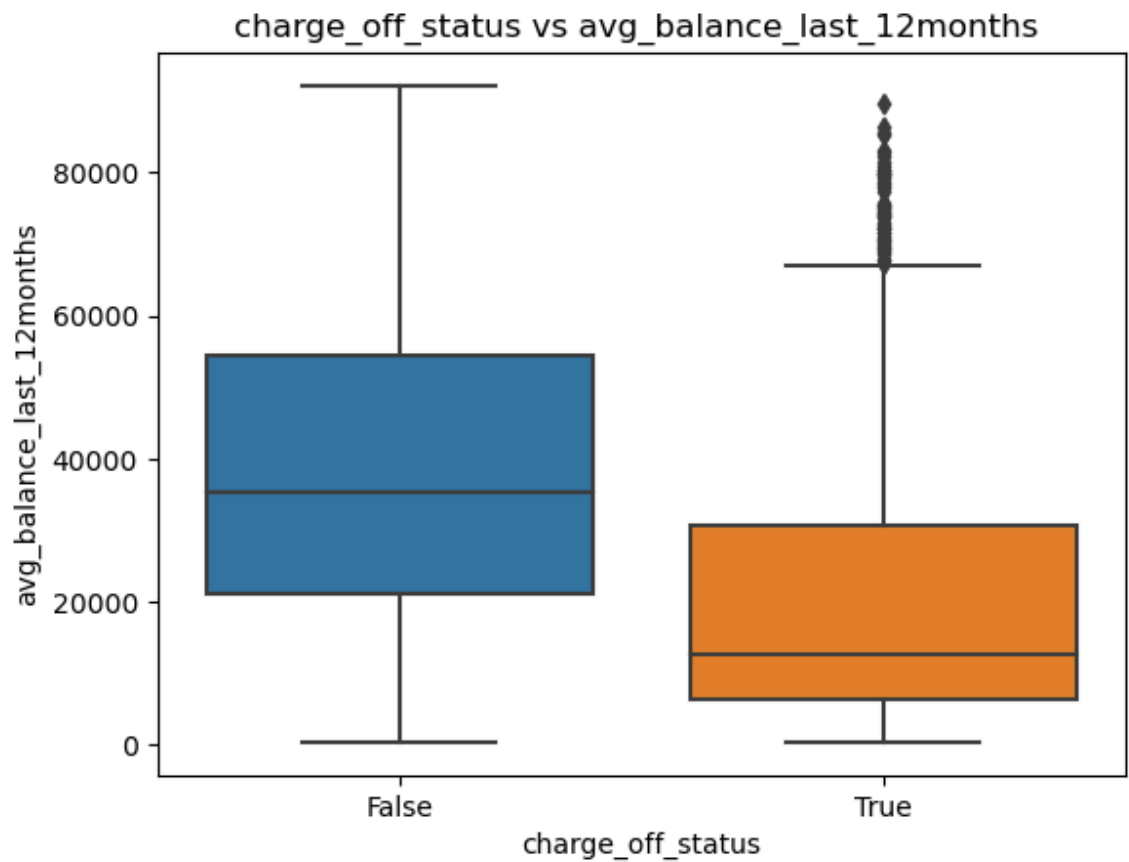
```
In [68]: outliers_avg_balance_last_12months = df[((df['avg_balance_last_12months'] -
outliers_avg_balance_last_12months
```

```
Out[68]:
```

account_open_date	age	location	occupation	income_level	fico_score	delinquency_status
-------------------	-----	----------	------------	--------------	------------	--------------------

0 rows × 27 columns

```
In [69]: sns.boxplot(x=df['charge_off_status'], y=df['avg_balance_last_12months'])  
plt.title('charge_off_status vs avg_balance_last_12months ' )  
plt.xlabel('charge_off_status')  
  
plt.show()
```



```
In [70]: # Filter data based on charge_off_status == 0
df_charge_off_0 = df[df['charge_off_status'] == 0]
df_charge_off_1 = df[df['charge_off_status'] == 1]

cap_value_0 = df_charge_off_0['avg_balance_last_12months'].quantile(0.95)
cap_value_1 = df_charge_off_1['avg_balance_last_12months'].quantile(0.95)

# Apply capping
df_charge_off_0['avg_balance_last_12months_capped'] = np.where(df_charge_off_0['avg_balance_last_12months'] > cap_value_0, cap_value_0, df_charge_off_0['avg_balance_last_12months'])
df_charge_off_1['avg_balance_last_12months_capped'] = np.where(df_charge_off_1['avg_balance_last_12months'] > cap_value_1, cap_value_1, df_charge_off_1['avg_balance_last_12months'])

# Concatenate the filtered dataframes
df = pd.concat([df_charge_off_0, df_charge_off_1])
df = df.reset_index(drop=True)
df
```

C:\Users\THUSHAN\AppData\Local\Temp\ipykernel_2856\1006456335.py:9: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy (https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

```
df_charge_off_0['avg_balance_last_12months_capped'] = np.where(df_charge_off_0['avg_balance_last_12months'] > cap_value_0, cap_value_0, df_charge_off_0['avg_balance_last_12months'])
```

C:\Users\THUSHAN\AppData\Local\Temp\ipykernel_2856\1006456335.py:10: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

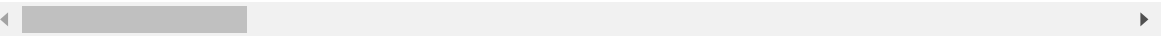
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy (https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

```
df_charge_off_1['avg_balance_last_12months_capped'] = np.where(df_charge_off_1['avg_balance_last_12months'] > cap_value_1, cap_value_1, df_charge_off_1['avg_balance_last_12months'])
```

Out[70]:

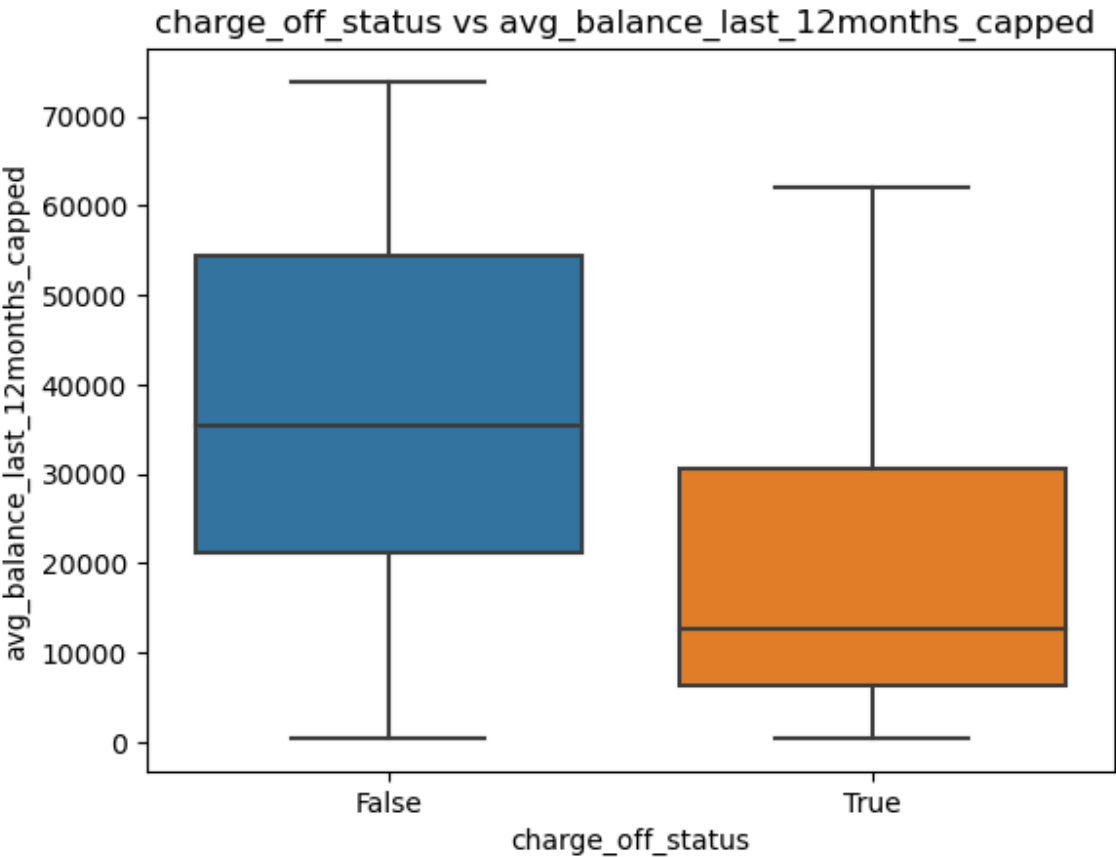
	account_open_date	age	location	occupation	income_level	fico_score	delinquency_s
0	2020-07-12	46	Miami	Engineer	71936	566.0	
1	2024-07-27	60	Houston	Lawyer	8574	787.0	
2	2022-03-06	25	Chicago	Teacher	59349	688.0	
3	2023-10-26	38	Los Angeles	Banker	6925	740.0	
4	2022-09-10	36	Los Angeles	Other	6197	758.0	
...	...	...	...	...	...	...	
6715	2021-01-17	65	San Francisco	Artist	10876	447.0	
6716	2024-09-15	29	Boston	Consultant	16347	469.0	
6717	2019-12-27	74	Chicago	Doctor	51012	580.0	
6718	2021-04-10	33	Boston	Consultant	14566	589.0	
6719	2024-06-20	68	Houston	Other	4501	321.0	

6720 rows × 28 columns

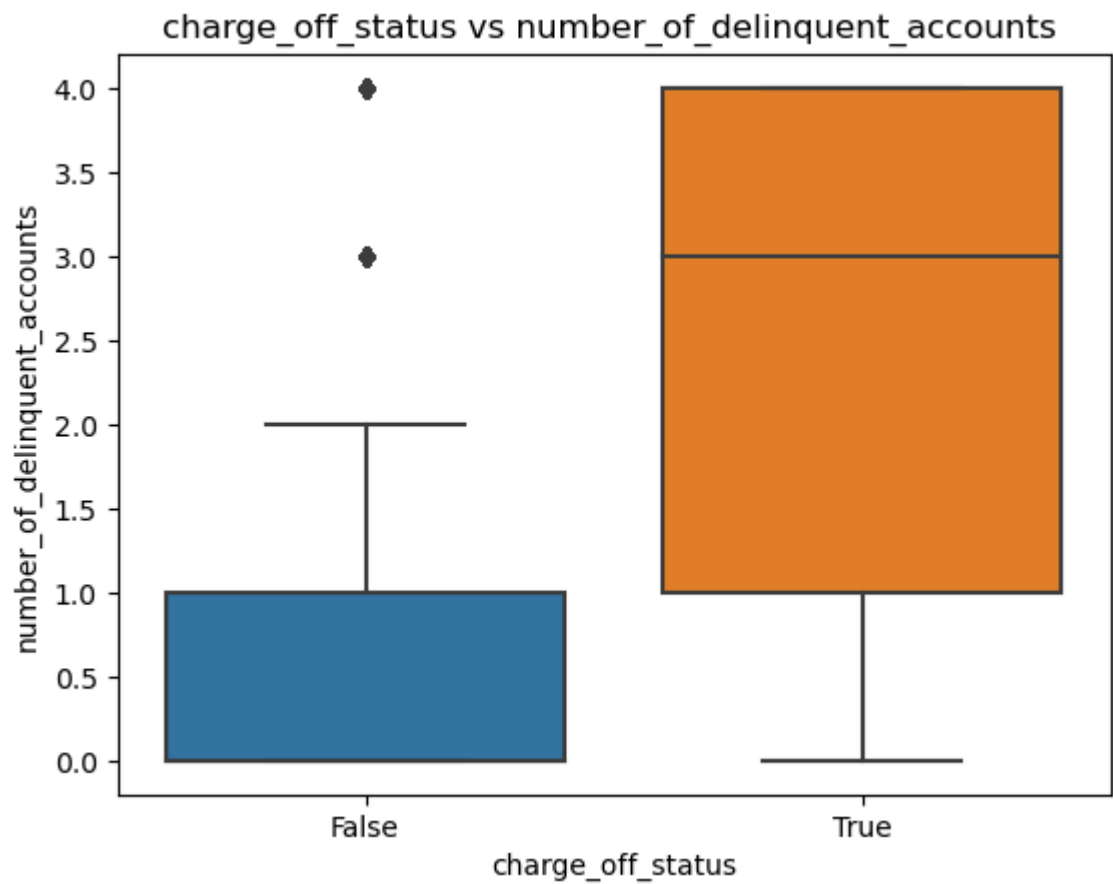


```
In [71]: sns.boxplot(x=df['charge_off_status'], y=df['avg_balance_last_12months_capped'],
plt.title('charge_off_status vs avg_balance_last_12months_capped ')
plt.xlabel('charge_off_status')

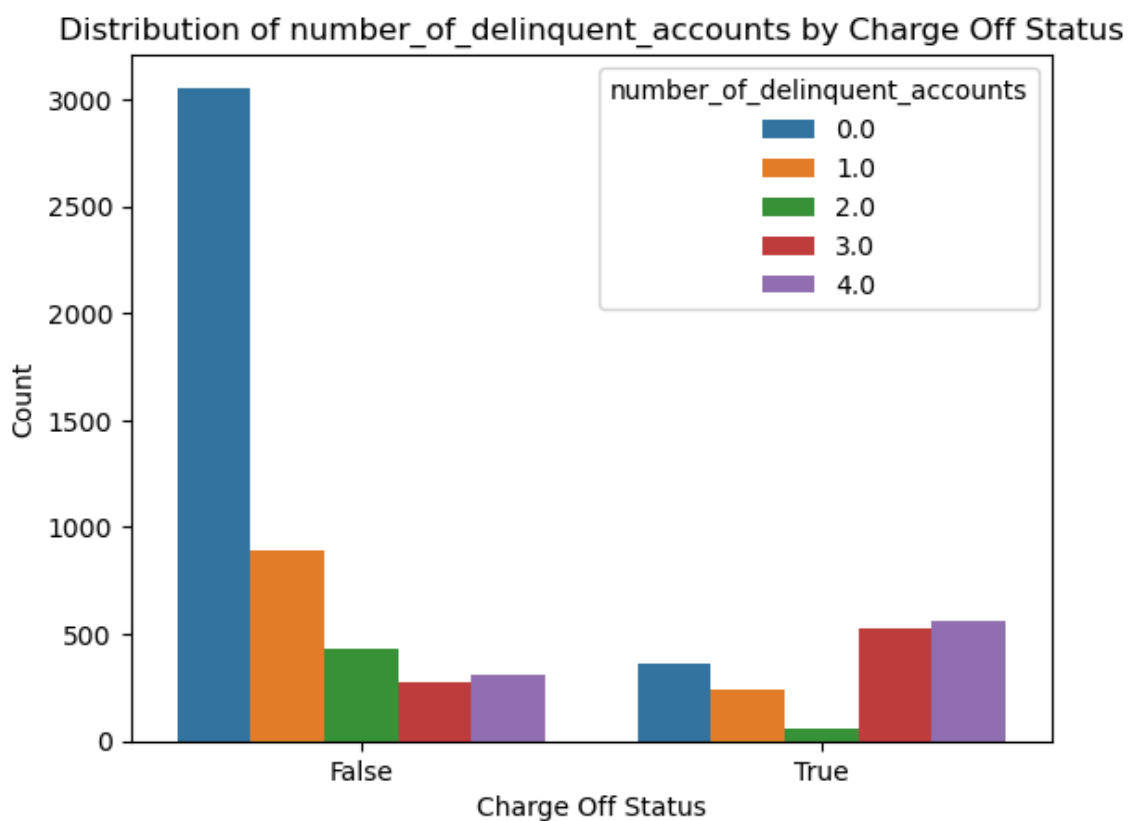
plt.show()
```



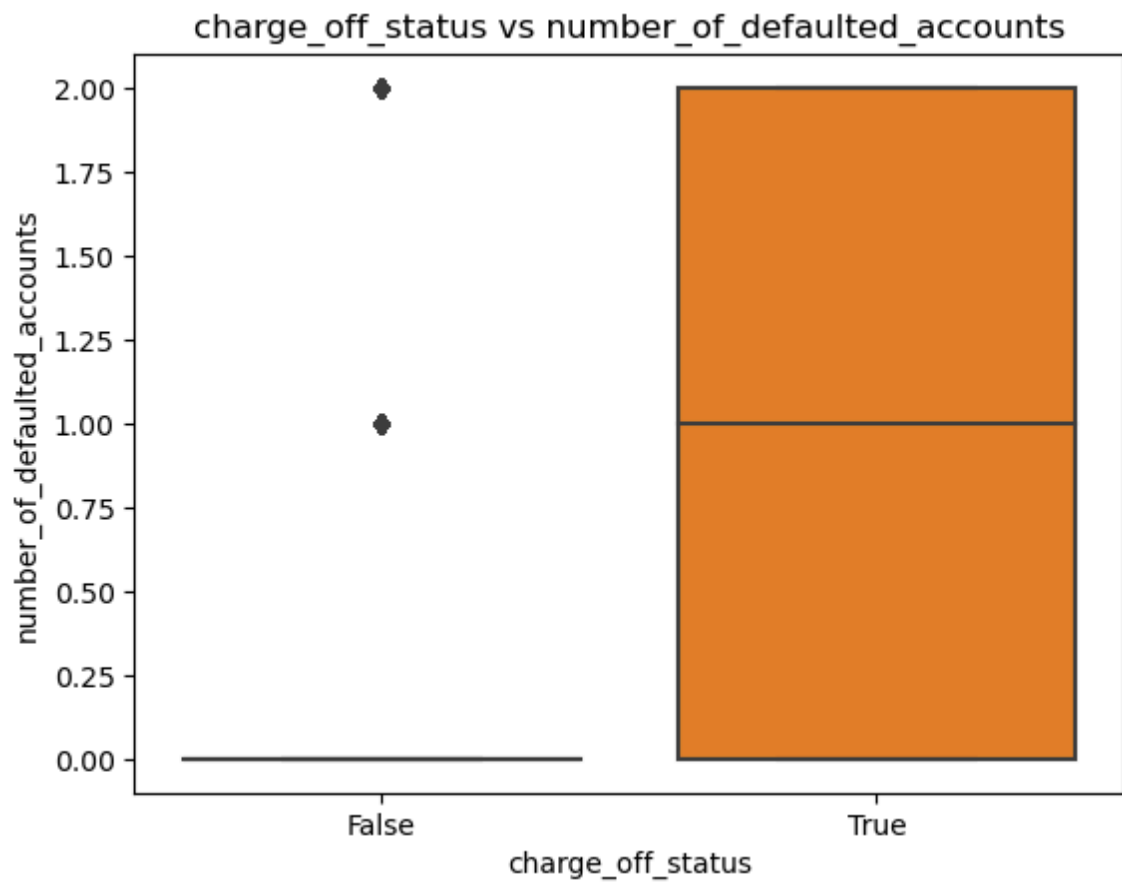
```
In [72]: sns.boxplot(x=df['charge_off_status'], y=df['number_of_delinquent_accounts'])  
plt.title('charge_off_status vs number_of_delinquent_accounts ' )  
plt.xlabel('charge_off_status')  
  
plt.show()
```



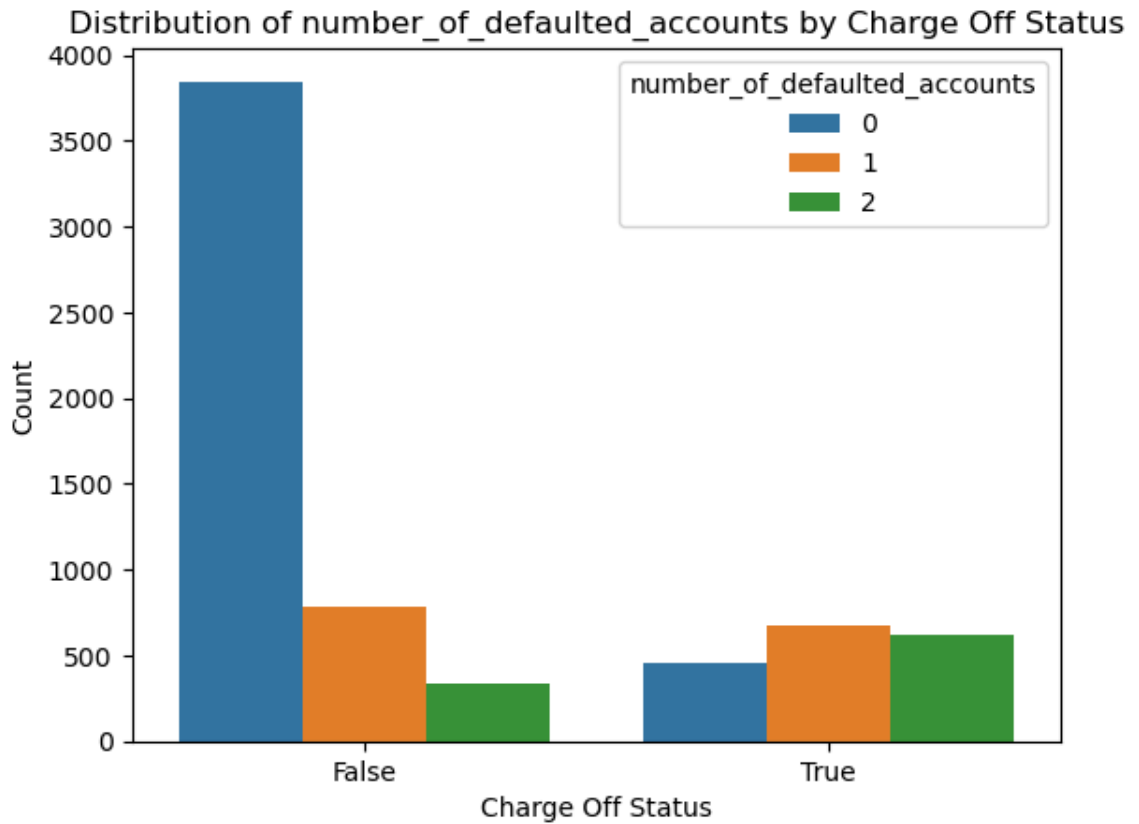
```
In [73]: sns.countplot(x='charge_off_status', hue='number_of_delinquent_accounts', data=df)
plt.title('Distribution of number_of_delinquent_accounts by Charge Off Status')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.show()
```



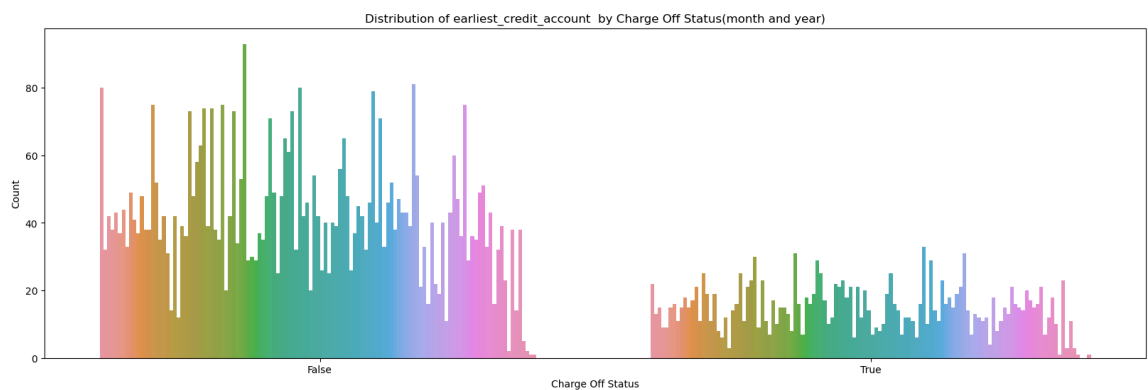
```
In [74]: sns.boxplot(x=df['charge_off_status'], y=df['number_of_defaulted_accounts'])  
plt.title('charge_off_status vs number_of_defaulted_accounts')  
plt.xlabel('charge_off_status')  
  
plt.show()
```



```
In [75]: sns.countplot(x='charge_off_status', hue='number_of_defaulted_accounts', data=df)
plt.title('Distribution of number_of_defaulted_accounts by Charge Off Status')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.show()
```



```
In [76]: plt.figure(figsize=(20, 6))
sns.countplot(x='charge_off_status', hue=df['earliest_credit_account'].dt.to_period('M').dt.to_timestamp(), data=df)
plt.title('Distribution of earliest_credit_account by Charge Off Status(month and year)')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.legend().remove()
plt.show()
```



In [77]:

```

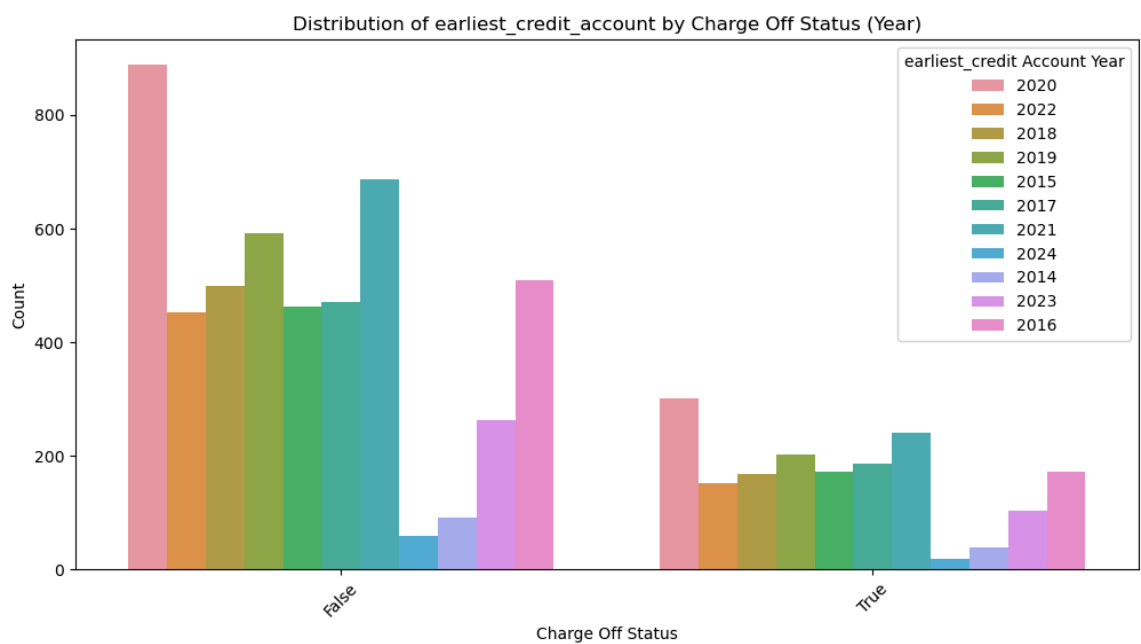
df_new=df.copy()
df_new['earliest_credit_account'] = pd.to_datetime(df['earliest_credit_accou

# Extract month and year and create a new column for plotting
df_new['credit_account_year'] = df_new['earliest_credit_account'].dt.to_per

plt.figure(figsize=(12, 6))
sns.countplot(x='charge_off_status', hue='credit_account_year', data=df_new)
plt.title('Distribution of earliest_credit_account by Charge Off Status (Year)')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.legend(title='earliest_credit Account Year')

plt.show()

```



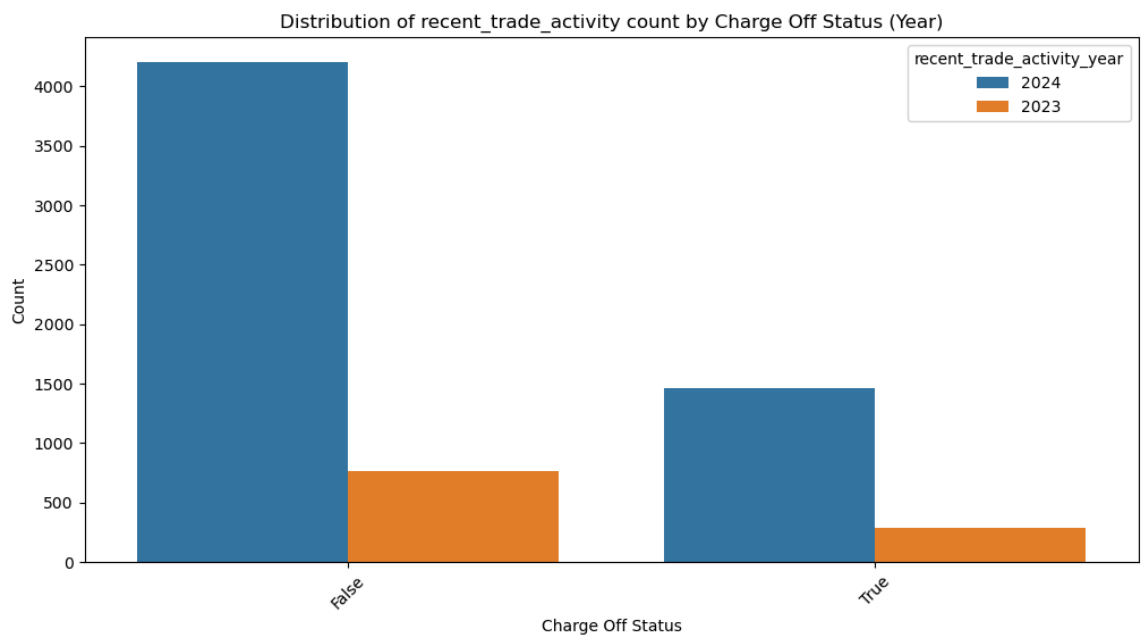
```
In [78]: df_new=df.copy()

df_new['recent_trade_activity'] = pd.to_datetime(df['recent_trade_activity'])

# Extract month and year and create a new column for plotting
df_new['recent_trade_activity_year'] = df_new['recent_trade_activity'].dt.to

plt.figure(figsize=(12, 6))
sns.countplot(x='charge_off_status', hue='recent_trade_activity_year', data=
plt.title('Distribution of recent_trade_activity count by Charge Off Status
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.legend(title='recent_trade_activity_year')

plt.show()
```

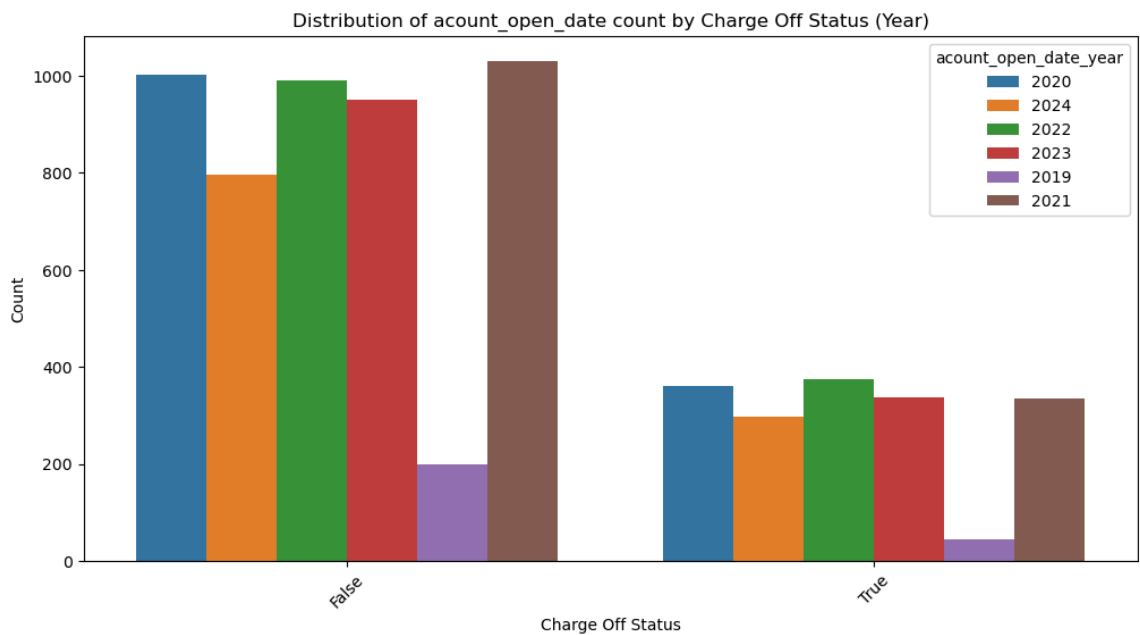


```
In [79]: df_new=df.copy()
df_new['account_open_date'] = pd.to_datetime(df['account_open_date'], errors='coerce')

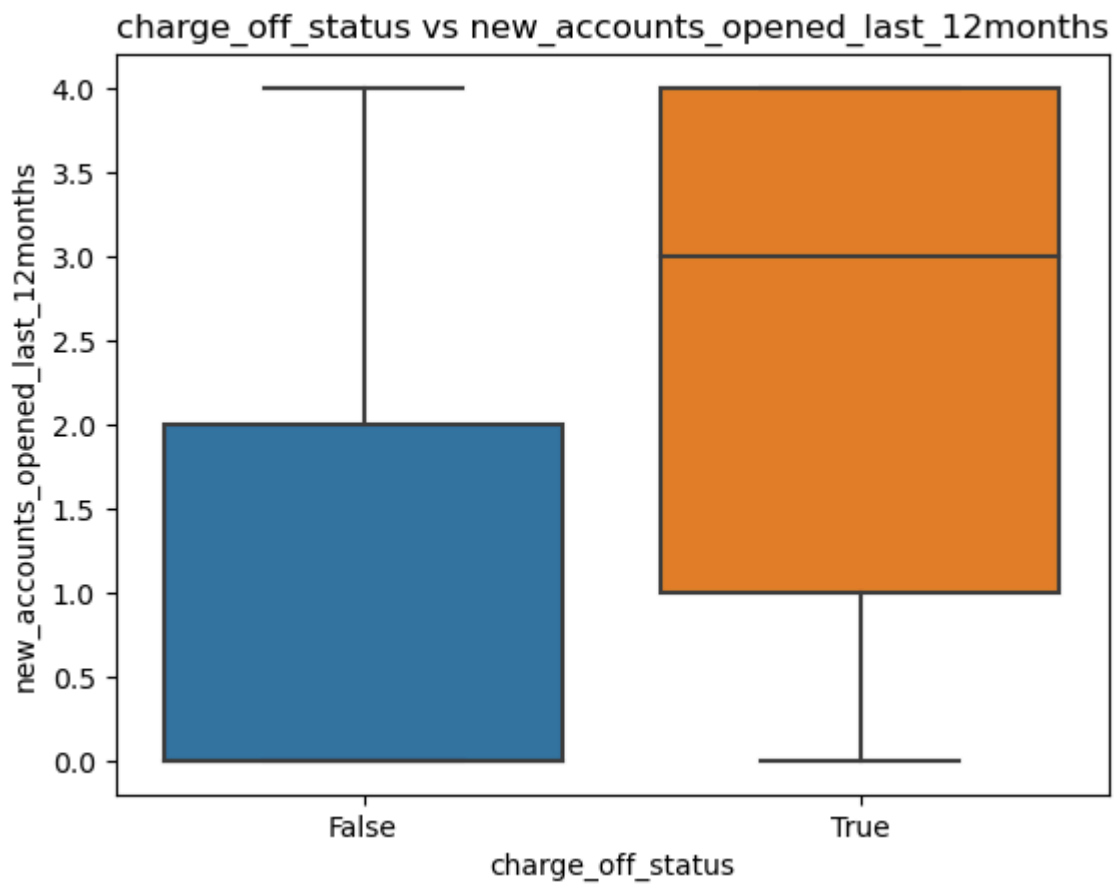
# Extract month and year and create a new column for plotting
df_new['account_open_date_year'] = df_new['account_open_date'].dt.to_period('Y')

plt.figure(figsize=(12, 6))
sns.countplot(x='charge_off_status', hue='account_open_date_year', data=df_new)
plt.title('Distribution of account_open_date count by Charge Off Status (Year)')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.legend(title='account_open_date_year')

plt.show()
```



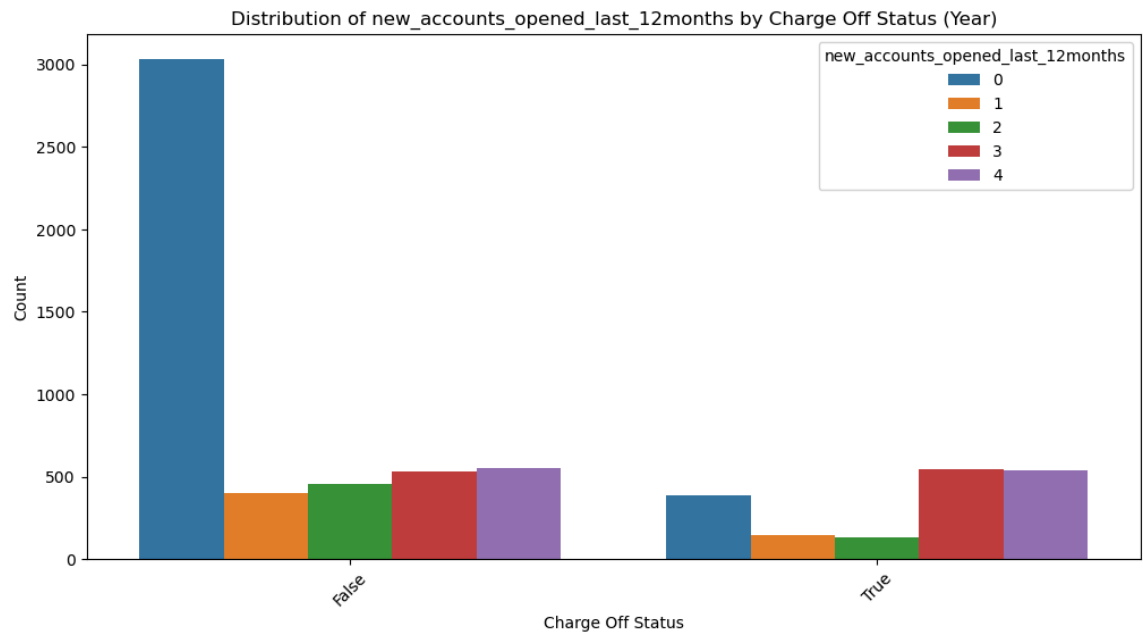
```
In [80]: sns.boxplot(x=df['charge_off_status'], y=df['new_accounts_opened_last_12months'])  
plt.title('charge_off_status vs new_accounts_opened_last_12months')  
plt.xlabel('charge_off_status')  
  
plt.show()
```



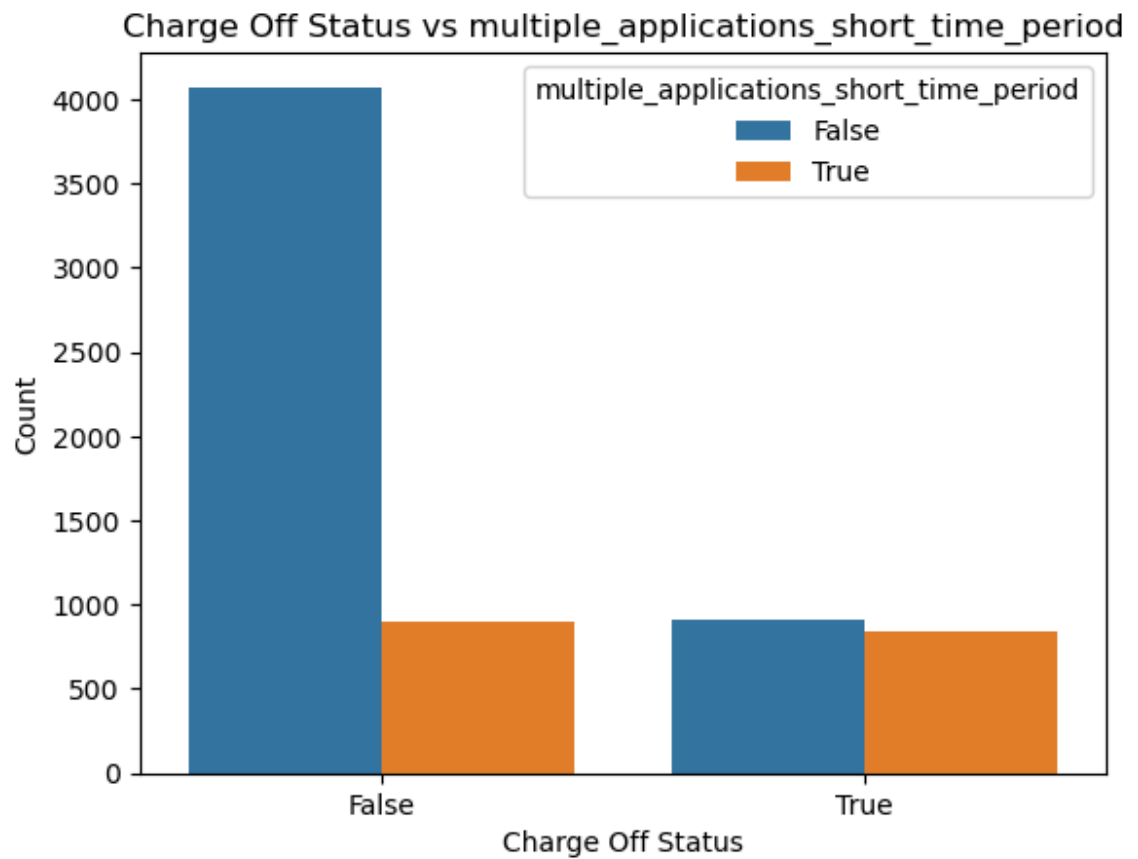
In [81]:

```
plt.figure(figsize=(12, 6))
sns.countplot(x='charge_off_status', hue='new_accounts_opened_last_12months')
plt.title('Distribution of new_accounts_opened_last_12months by Charge Off Status')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.xticks(rotation=45)

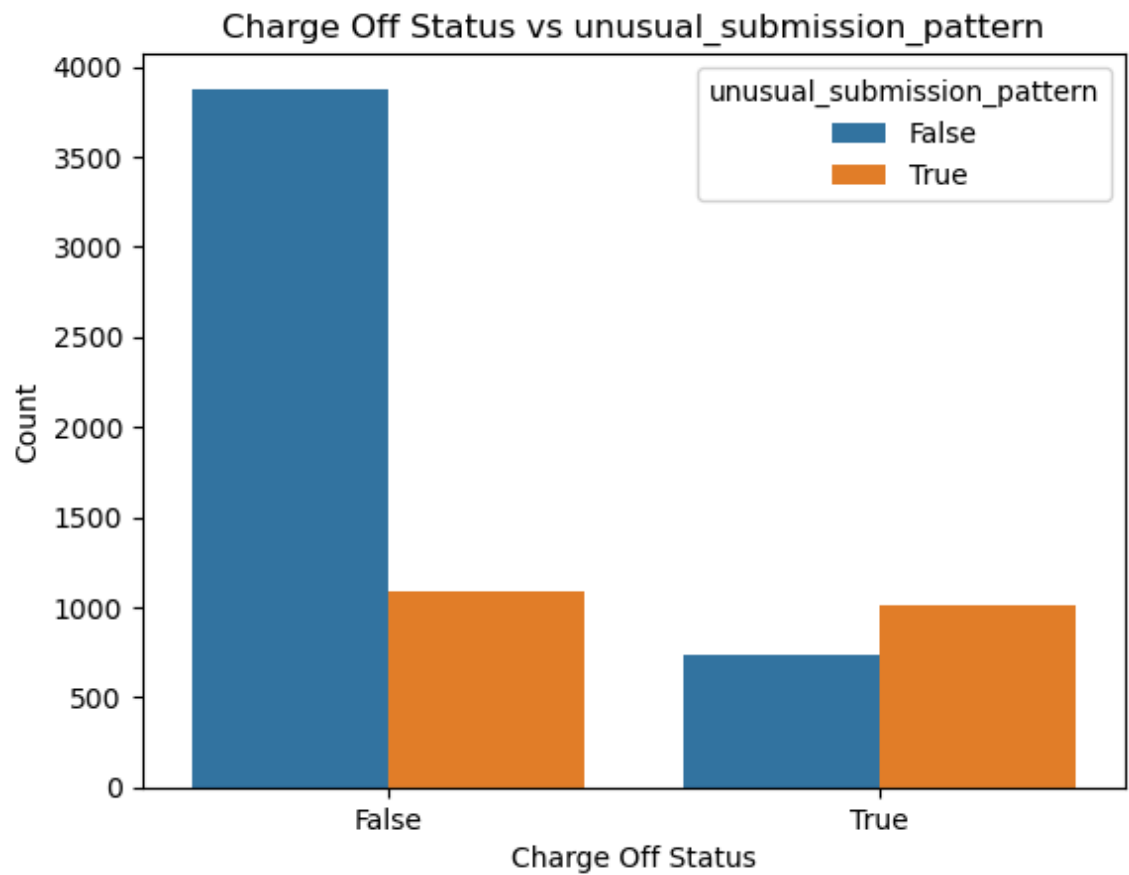
plt.show()
```



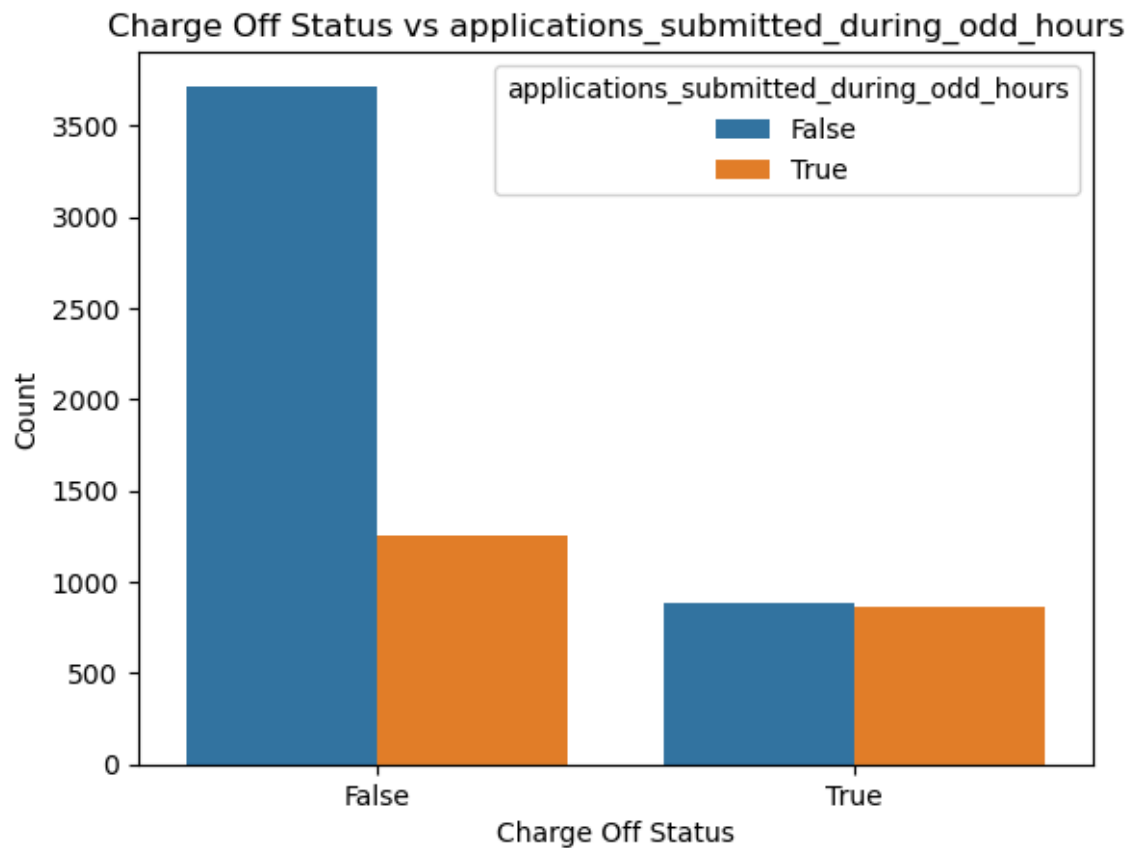
```
In [82]: sns.countplot(x='charge_off_status', hue='multiple_applications_short_time_p',  
plt.title('Charge Off Status vs multiple_applications_short_time_period')  
plt.xlabel('Charge Off Status')  
plt.ylabel('Count')  
plt.show()
```



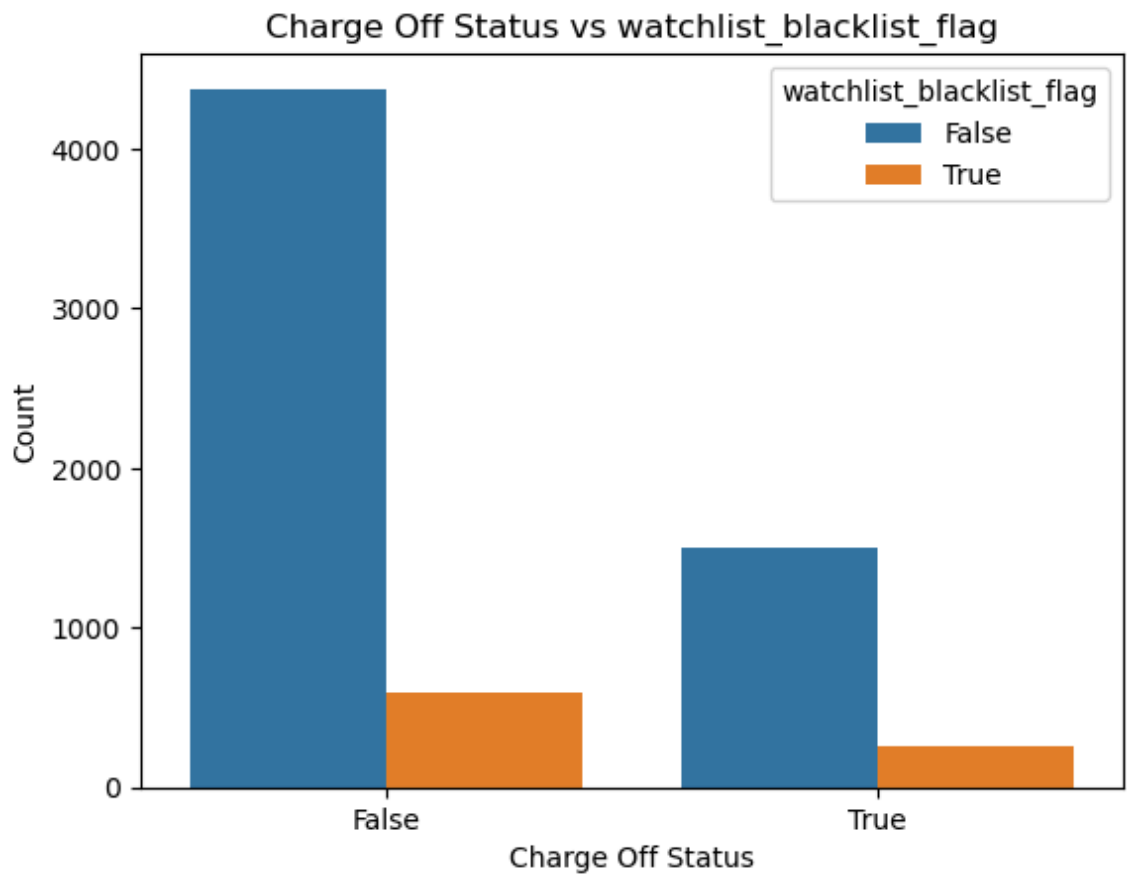
```
In [83]: sns.countplot(x='charge_off_status', hue='unusual_submission_pattern', data=
plt.title('Charge Off Status vs unusual_submission_pattern')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.show()
```



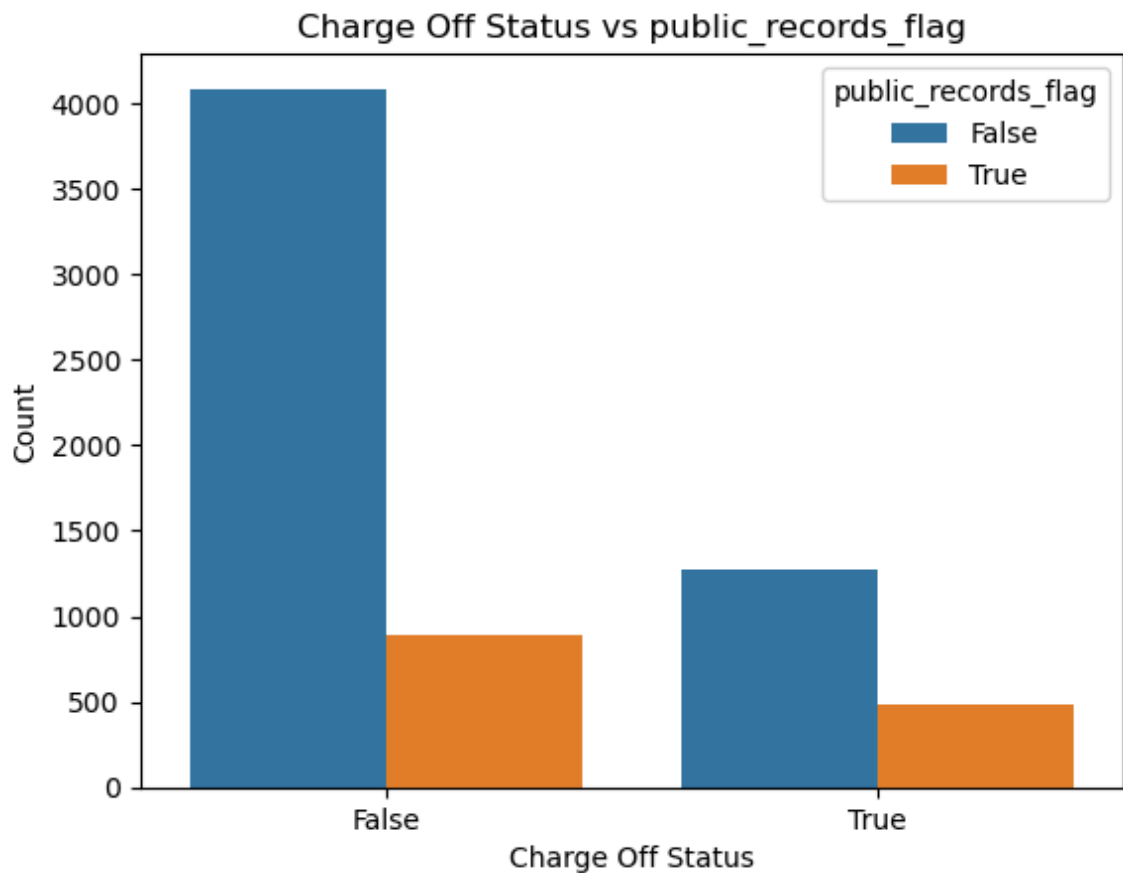
```
In [84]: sns.countplot(x='charge_off_status', hue='applications_submitted_during_odd_
plt.title('Charge Off Status vs applications_submitted_during_odd_hours')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.show()
```



```
In [85]: sns.countplot(x='charge_off_status', hue='watchlist_blacklist_flag', data=d  
plt.title('Charge Off Status vs watchlist_blacklist_flag')  
plt.xlabel('Charge Off Status')  
plt.ylabel('Count')  
plt.show()
```



```
In [86]: sns.countplot(x='charge_off_status', hue='public_records_flag', data=df)
plt.title('Charge Off Status vs public_records_flag')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.show()
```



```
In [87]: df
```

```
Out[87]:
```

	account_open_date	age	location	occupation	income_level	fico_score	delinquency_s
0	2020-07-12	46	Miami	Engineer	71936	566.0	
1	2024-07-27	60	Houston	Lawyer	8574	787.0	
2	2022-03-06	25	Chicago	Teacher	59349	688.0	
3	2023-10-26	38	Los Angeles	Banker	6925	740.0	
4	2022-09-10	36	Los Angeles	Other	6197	758.0	
...	...	...	...	...	...	...	
6715	2021-01-17	65	San Francisco	Artist	10876	447.0	
6716	2024-09-15	29	Boston	Consultant	16347	469.0	
6717	2019-12-27	74	Chicago	Doctor	51012	580.0	
6718	2021-04-10	33	Boston	Consultant	14566	589.0	
6719	2024-06-20	68	Houston	Other	4501	321.0	

6720 rows × 28 columns

```
In [88]: df['charge_off_status'] =df.pop('charge_off_status')
```

```
In [89]: df.dtypes
```

```
Out[89]: account_open_date      datetime64[ns]
age                             int64
location                       object
occupation                    object
income_level                   int64
fico_score                    float64
delinquency_status            int64
number_of_credit_applications  int64
debt_to_income_ratio          float64
payment_methods_high_risk      bool
max_balance                   float64
avg_balance_last_12months      float64
number_of_delinquent_accounts float64
number_of_defaulted_accounts  int64
earliest_credit_account        datetime64[ns]
recent_trade_activity          datetime64[ns]
new_accounts_opened_last_12months int64
multiple_applications_short_time_period bool
unusual_submission_pattern     bool
applications_submitted_during_odd_hours bool
watchlist_blacklist_flag       bool
public_records_flag            bool
date_contradiction_flag        bool
delinquency_status_log         float64
number_of_credit_applications_log float64
max_balance_log                float64
avg_balance_last_12months_capped float64
charge_off_status              bool
dtype: object
```

```
In [90]: df_clean = df.drop(['delinquency_status', 'number_of_credit_applications'],
```

In [91]: `df_clean.dtypes`

```
Out[91]: account_open_date      datetime64[ns]
age                          int64
location                     object
occupation                   object
income_level                 int64
fico_score                  float64
debt_to_income_ratio        float64
payment_methods_high_risk    bool
number_of_delinquent_accounts float64
number_of_defaulted_accounts int64
earliest_credit_account      datetime64[ns]
recent_trade_activity        datetime64[ns]
new_accounts_opened_last_12months int64
multiple_applications_short_time_period bool
unusual_submission_pattern   bool
applications_submitted_during_odd_hours bool
watchlist_blacklist_flag     bool
public_records_flag          bool
date_contradiction_flag      bool
delinquency_status_log       float64
number_of_credit_applications_log float64
max_balance_log              float64
avg_balance_last_12months_capped float64
charge_off_status            bool
dtype: object
```

In [92]: `df_clean`

```
Out[92]:
```

	account_open_date	age	location	occupation	income_level	fico_score	debt_to_incor
0	2020-07-12	46	Miami	Engineer	71936	566.0	1
1	2024-07-27	60	Houston	Lawyer	8574	787.0	1
2	2022-03-06	25	Chicago	Teacher	59349	688.0	1
3	2023-10-26	38	Los Angeles	Banker	6925	740.0	1
4	2022-09-10	36	Los Angeles	Other	6197	758.0	1
...	...	...	...	...	...	...	...
6715	2021-01-17	65	San Francisco	Artist	10876	447.0	2
6716	2024-09-15	29	Boston	Consultant	16347	469.0	1
6717	2019-12-27	74	Chicago	Doctor	51012	580.0	2
6718	2021-04-10	33	Boston	Consultant	14566	589.0	1
6719	2024-06-20	68	Houston	Other	4501	321.0	1

6720 rows × 24 columns

In [148]: `df_clean .to_csv('df_cleaned.csv', index=False)`

In []:

In []: